

1 Unit-Testing (Task 4)

The unit-testing performed, throughout the development process, focused on public methods of the “LidDrivenCavity” class (LDC), and treated the “SolverCG” class (SCG), as a unit onto itself. The unit tests kept the same checking conditions when extending the original serial code with MPI and OpenMP, however significant refactoring was necessary to facilitate these equivalent checks. The source code for these tests can be found in Appendix B.

1.1 “LidDrivenCavity.h”

The tests written for the LDC were designed to validate the public methods the user would be accessing within the main script. The setters and getters such as “SetDomainSize” were not tested due to their simplistic nature. If more complexity was used in the aforementioned methods then they also would have been tested.

1.1.1 LidDrivenCavity::PrintConfiguration()

The “PrintConfiguration” test utilised a stream-buffer as shown in Code Snippet B.2. After rerouting the output stream, the printed information is extracted using a pattern search, via a regular expression. Finally, a vector is stored contained the printed values and the expected value. Using the “compareSolution()” defined in Code Snippet B.6 the 2-norm or the error is computed and checked against a tolerance of $1e-3$.

1.1.2 LidDrivenCavity::WriteSolution()

On the other hand the “WriteSolution” test involved conditions that were agnostic of the accuracy of the SCG class. The conditions require that the solver has successfully written out to the file requested through a comprehensive list of checks. These range from the file existing; the maximum number of values on a given line being the six parameters: x co-ordinate, y co-ordinate, vorticity, stream function, x velocity and y velocity; and finally that there are the correct number of lines / grid points being printed.

1.2 “SolverCG.h”

Whilst the previous unit tests weren’t concerned with the accuracy of the data being calculated, the following tests validate the “SolverCG::Solve” method. Using the below equation Equation 1 that spatial Poisson problem was defined.

$$-\nabla^2 \psi = \omega \quad (1)$$

The main way the test changed once MPI had been implemented, was that the local error between the numerical and analytical solutions had to be precomputed and *reduced* to every other rank. The tolerance used in the following tests was equivalent to the one set in the main solver file to maintain consistency.

1.2.1 SolverCG::Solve() - Analytical

Using a sinusoidal solution allowed a varying frequency in the stream function, shown in Equation 2. The significance of this function was that the boundary conditions at the walls of the cavity were enforced given that at $x = L_x$, $y = L_y$ and at the origin.

$$\psi = \sin(\pi * k * x) \sin(\pi * l * y), \quad k = \frac{2\pi}{L_x}, \quad l = \frac{2\pi}{L_y} \quad (2)$$

Combining Equations 1 and 2, gave the input solution to the “Solve” method as it used a conjugate gradient algorithm to solve for x in $Ax = B$. This input is found below in Equation 3.

$$\omega = \pi^2 (k^2 + l^2) \sin(\pi * k * x) \sin(\pi * l * y), \quad k = \frac{2\pi}{L_x}, \quad l = \frac{2\pi}{L_y} \quad (3)$$

As seen in Code Snippet B.5 the domain used was of size 6,000,000 grid points and could be run with a variable number of MPI ranks as shown in Code Snippet B.6 through the Boost global fixture.

1.2.2 SolverCG::Solve() - Zero Input

Constrastingly, the “Zero Input” test was used to ensure that during the development of the parallelisation, the solver still gave back a zero vector. This functionality was implemented to avoid errors from failing to converge with such a small vorticity input.

2 Extending code with MPI (Task 5)

2.1 Parallel Vs. Serial Performance

After having implmented the MPI methods into the serial solver the performance was plotted in Figure 1 below.

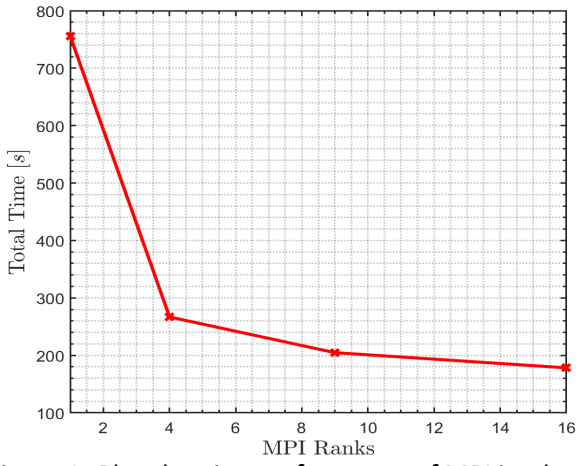


Figure 1: Plot showing performance of MPI implementation against serial case

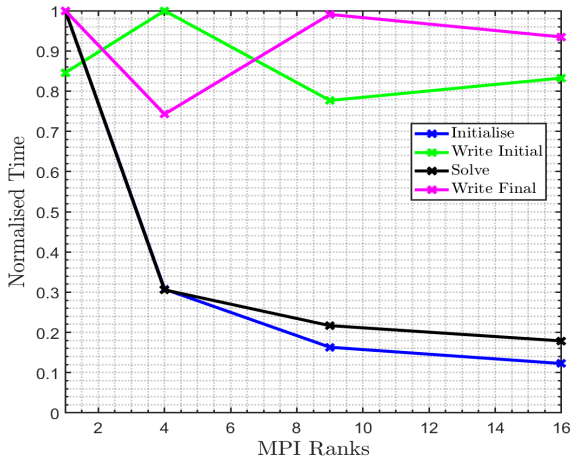
Table 1: Settings used when investigating parallel vs. serial performance

Profile	Lx	Ly	Nx	Ny	Re	dt	T
Large	1.00	1.00	201	201	1000	0.005	5.000
Small	1.00	1.00	101	101	500	0.005	5.000

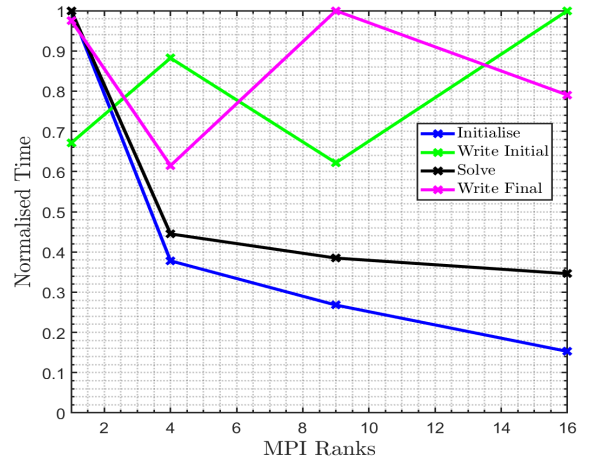
The scaling plot above was produced with the “Large” settings found above in Table 1. These parameters were chosen to ensure a large enough domain to take advantage of the number of ranks available. The main setting change of note for a given profile was highlighted above.

Through testing at other settings, similar plots were made (found in Appendix A), and it was seen that the smaller the spatial domain the worse the scaling appeared. This was due to the increase in relative communication time between neighbour ranks. When a smaller grid was chosen, more time was spent communicating between ranks relative to actually doing computation on their local vectors. Furthermore, from the figure it can be seen that the increase in ranks from serial case does flatten off in terms of reduction in total time. This will be due to both the domain size, not utilising the maximum number of ranks (16) as efficiently as possible; and primarily, that the communication overhead through the increase in calls to exchange information between neighbour ranks (seen in Code Snippet C.2) became detrimental to performance.

To compare the relative reductions in timings of the main components of the main program, a normalised time reduction scaling plot was produced, as seen below in Figure 2.



(a) MPI (Large)



(b) MPI (Small)

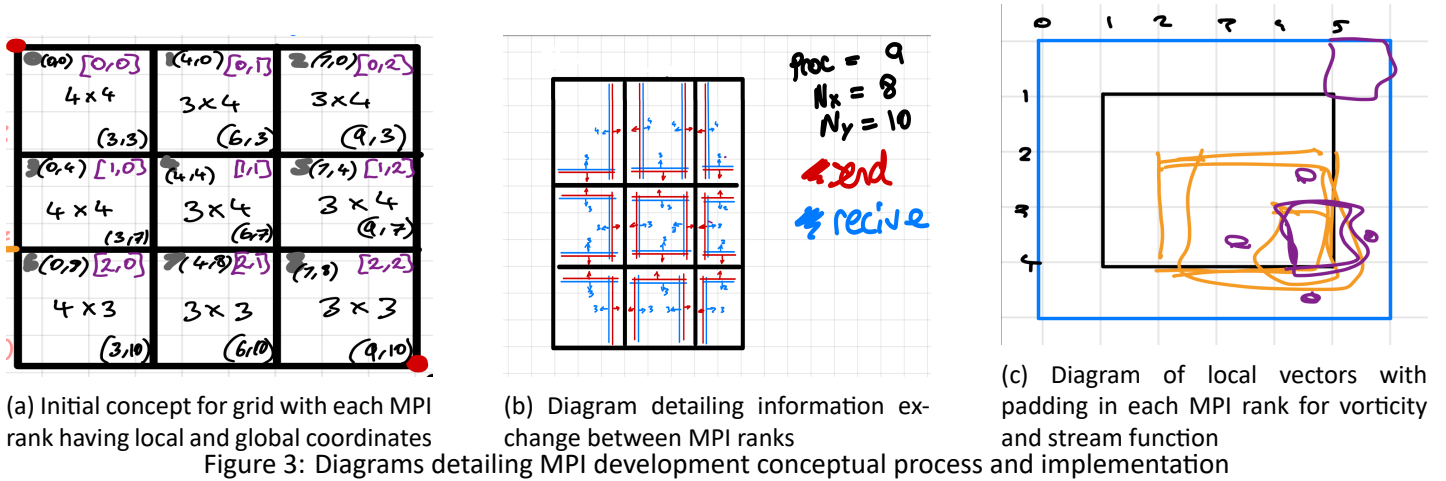
Figure 2: Plots showing normalized time scaling plot to see the effect MPI parallelisation has on both “Large” and “Small” domains

As mentioned earlier its clear that the reduction in solve time was greater with the larger domain. From Figure 2a we see a 82% reduction in “Solve” time with 9 ranks compared to the serial case. However, for the “Small” parameters, Figure 2b respectively shows 64 % reduction. Furthermore, this plot also shows a similar level of gains in the across both settings with the “Initialise” function which was used when allocating the local vectors with dynamic memory and creating the “dataGrid” objects. Shown in both figures, it is clear that the writing to “.txt” file operations maintained a constant time within increasing rank number, accounting for variance. This was due to the fact that when converting this operation from serial to parallel, it was deemed efficient to instead synchronously iterate over the global grid and only print the associated data from the correct rank. This approach was taken in order to avoid regathering all data, rendering the distribution of work / memory pointless.

2.2 Design Methodology

The source code for the grid implementation can be found in the “prl::dataGrid” struct in Appendix C. The extending of the serial code to utilise a square number of MPI ranks, primarily focused on the implementation of a virtual topology.

As seen in Figure 3a, the global grid defined by the L_x, L_y, N_x and N_y parameters had to be divided as evenly as possible to ensure the load was balanced and the computations were distributed as evenly as possible amongst the ranks. Using the logic found in the method “gridData::init”, the grid was divided into chunks along both dimensions.



Moreover, when initialising the grid, each rank with the use of the “gridData” object could store information such as its global coordinates for its top-left and bottom-right corners and its neighbour ranks. This data was extremely useful when the exchange of edge values became necessary between neighbour ranks, which can be seen in Code Snippet C.2. As seen in Figure 3b, each rank had to know to only send to an available neighbour and send and data with the same size dimensions. This was implemented using non-blocking send and receive messages (“MPI_Isend” and “MPI_Irecv”) to ensure there was no wasted time between sending to the neighbour above and to the right, as the information didnt depend on each other. Shown in Figure 3c, each local vector was actually stored with padding to allow receives of neighbour data, and to keep numerical scheme functions consistent. However, to ensure that the current rank had the information necessary to move on in the program, a “MPI_WAITALL” was implemented utilising “MPI_REQUEST” objects. Furthermore, to ensure all resources created by MPI were deallocated, “MPI_REQUEST_FREE” calls were implemented for the send requests, whilst the wait function freed the receive requests automatically.

2.2.1 LidDrivenCavity

The main efforts to parallelise the code in the LDC class, was to work on the “Advance” method. Here when operating on the vorticity vector, the parallel code could perfrom well. Through the use of checks whether the current rank had no neighbour in a direction implied whether the rank was at a wall. Therefore, when computing different schemes based of that information, the rank only operated on the correct data. For example, when considering the “interior” points schemes with calculating and advancing vorticity, the rank had to check if they were at walls and such only operate on their own local “interior” data. This was acheived through a ternary operator in the inital and final loop counter values, shown in Code Snippet D.1.

2.2.2 SolverCG

On the other hand, within the SCG class, there were much more opportunities to take advantage of the distribution of work. Significantly, all the function calls using BLAS were improved due to their reduced vector size. Furthermore, the operations involving imposing boundary conditions, and applying the matrix operation of Ax were significantly imprved as the size of the for loops reduced as the number of ranks increased. One thing to note was that the calculations using parameters like α and β had to be locally computed and reduced to all ranks before being used, to maintain the same serial logic as before.

3 Extending code with OpenMP (Task 6)

3.1 Parallel Vs. Serial Performance

It can be seen that generally the parallelisation was implemented correctly as the total time during execution was reduced as the number of “OMP” threads increasedbelow in Figure 4. Interestingly, the OpenMP solver can be seen to actually become significantly worse after roughly 8 threads. This was hypothesised due to the the overhead between the threads when encountering for loops and importantly caching issues. Expanding on this further, when the number of threads increased the amount of cache used did also; therefore after a certain point the executable no onger runs as efficiently as possible.

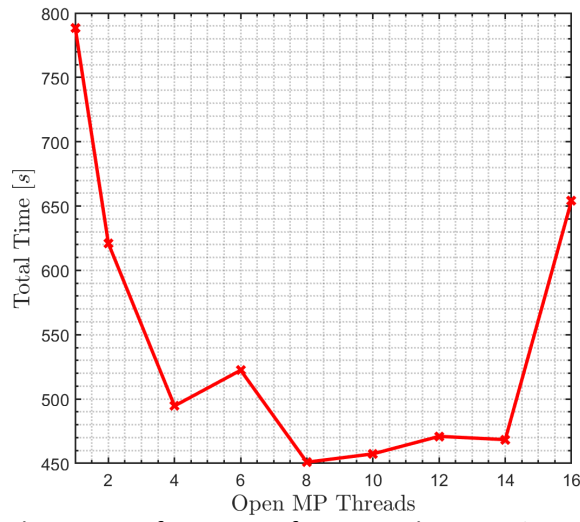
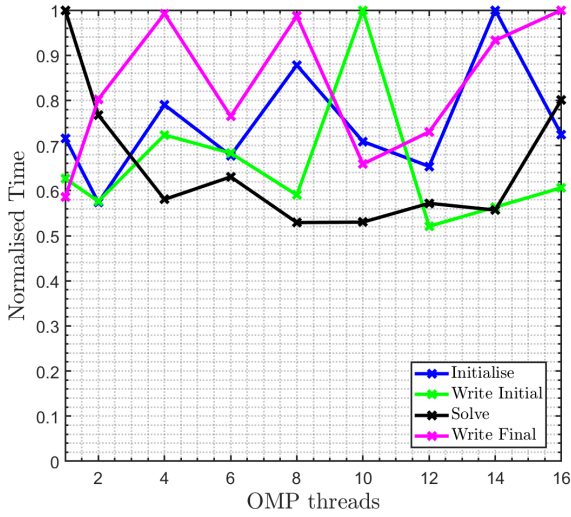
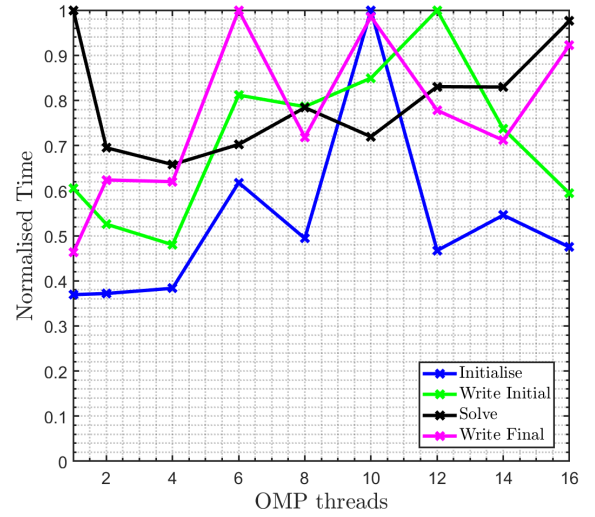


Figure 4: Plot showing performance of OMP implementation against serial case

As before, the normalised time plot has been produced below in Figure 5.



(a) OMP (Large)



(b) OMP (Small)

Figure 5: Plots showing normalized time scaling plot to see the effect Open MP parallelisation has on both “Large” and “Small” domains

It is apparant that when smaller domain settings were used the solver did not show very good scaling. This is in contrast to the MPI implementation where, even at “Small” settings the running time still reduced. Specifically, there was a 48% reduction in “Solve” time when increasing the serial case to have up to 8 threads. Furthermore, unlike the MPI case, the “Write” time was severely increased with thread number showing major overhead in communication with threads hanging about and not doing work. This makes sense however as there was a MPI barrier within this function that would force sequential writes for the ordered output.

3.2 Design Methodology

When implementing Open MP for the initial design, one major design concept was used. This utilised the “#pragma omp parallel for schedule(static)” pattern on almost all for loops where possible, ensuring no race conditions or false sharing occurred. This process however clearly was not the most effective as the performance did not scale as well as the MPI case with thread count (and rank count).

4 Optimising with profiler (Task 7)

In order to profile the code that had been developed so far, the Oracle Developer Studio was used. During testing, the code was run in serial with a single-rank and single-thread configuration. For more obvious reductions in timings to be seen, the simulation was run with the following paramters: `--Lx 1 --Ly 1 --Nx 301 --Ny 301 --T 1 --dt 0.005 --Re 2000`.

The results from the profiling task are shown below in Table 2. From running the serial code, it was noted that the main time spent was within the “Solve” method on the SCG class, as seen by Figure 6.

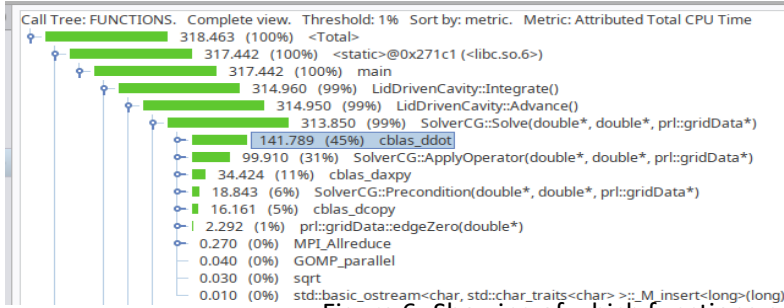


Figure 6: Showing of which functions program spends most time in

Looking further it could be seen that through the use of BLAS the 2nd nested for loop could be removed from both the “Precondition” and “ApplyOperator” method. From Table 2, it can be seen that the changes implemented in the aforementioned methods, shown in Code Snippet E.1, resulted in reductions in function (inclusive) time of 42% and 74% respectively. This was achieved through a strategic use of “DAXPY” and “DSCAL” methods. This approach was used because the program can now compute the entire row needed in these methods as opposed to iterating through the entire grid and reevaluating the same values multiple times.

Table 2: Shows changes to methods and reduction in timings, evidenced by Figure A.3

<i>Function</i>	<i>Change</i>	<i>Total (Inclusive-CPU) Time before</i>	<i>Total (Inclusive-CPU) Time after</i>	<i>Function (Inclusive) Time before</i>	<i>Function (Inclusive) Time after</i>
ApplyOperator	Replaced interior nested for loop with cblas	26.699	21.395	11.858	6.825
Precondition	Replaced interior nested for loop with cblas	25.208	21.355	5.214	1.381
Advance	Replaced interior nested for loop with cblas for current vorticity	41.079	40.008	39.598	38.777
Advance	Replaced interior nested for loop with cblas to advance vorticity	40.028	40.789	38.677	39.948
All Changes	Implemented changes above compared to none	456.83	317.892	-	-

On the other hand when the same approach was taken within the LDC class, the results were not as expected. In the best case, the reduction in function time was 2% and at worst caused an increase in function time of 3%. This can be noted due to the complexity of the algorithm needed to match the logic of the previous code. In order to evaluate the scheme which advances vorticity, over 12 lines of BLAS function calls had to be made to match this, as shown in Code Snippet D.1. In the future if the simulation was run on a larger domain problem then a more modest result could be expected.

Overall, once the simulation had been run again with one MPI rank and one Open MP thread but at larger time domain, the performance improvements from profiling were clear. All the changes when implemented compared to the base case after having just implemented Open MP, led to a reduction in overall inclusive CPU time of 30%.

5 Summary

In conclusion, because of the distributed nature of MPI programming, and given that the problem at hand is a large spatial Poisson problem that is being integrated through time, it is clear why this methodology achieved much better reductions in timings as the number of resources (ranks) increased. On the other hand, due to the nature of multi-threaded programming needing more optimisations when it comes to accessing the cache, avoiding race conditions and false sharing; it is possible that the executable was not as optimised to take advantage of multiple threads. Both of these conclusions were drawn from their associated scaling plot and through development and testing.

Supporting Diagram

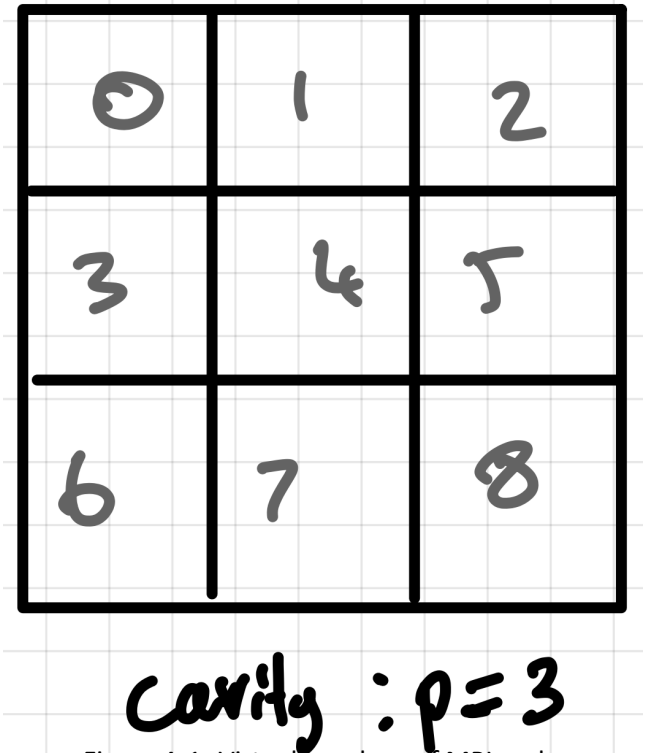
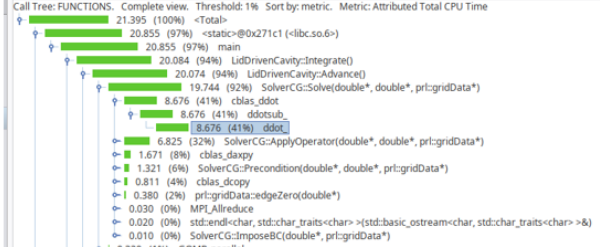
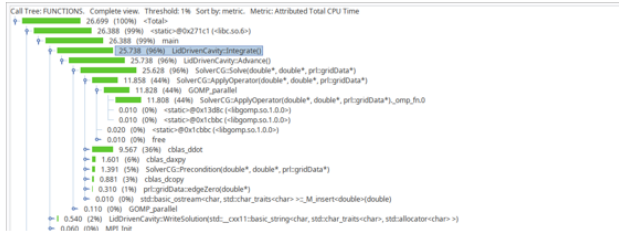


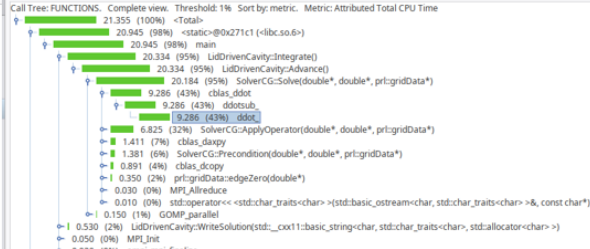
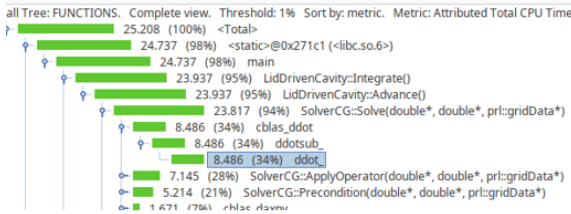
Figure A.1: Virtual topology of MPI ranks

Figure A.2: “timings.txt” to show sample data for scaling plot

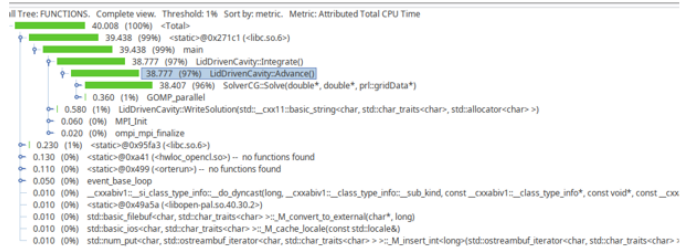
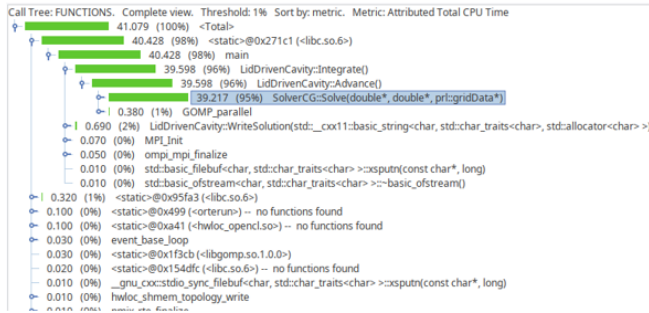
ApplyOperator



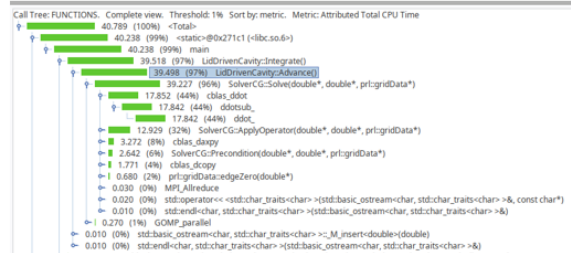
Precondition



Current Interior Vorticity



Advance Interior Vorticity



All Changes

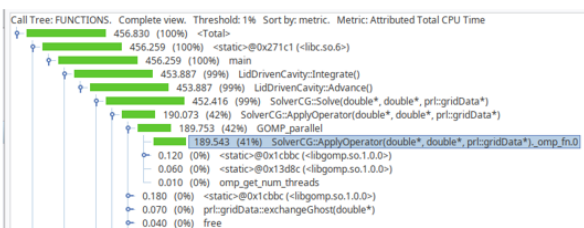
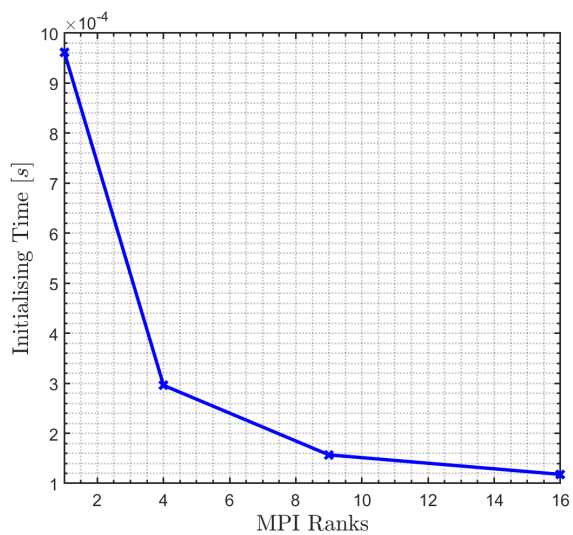
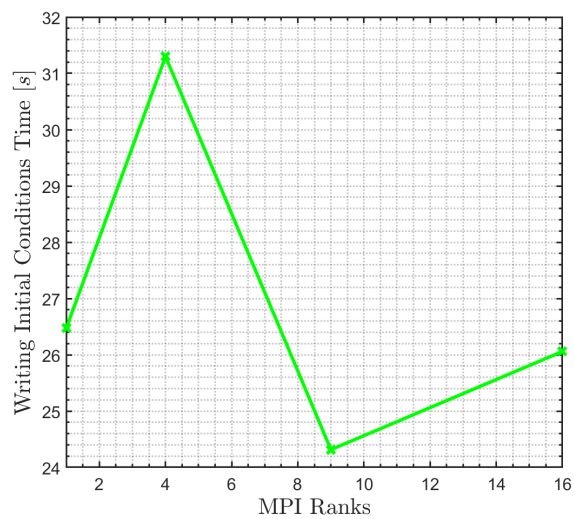


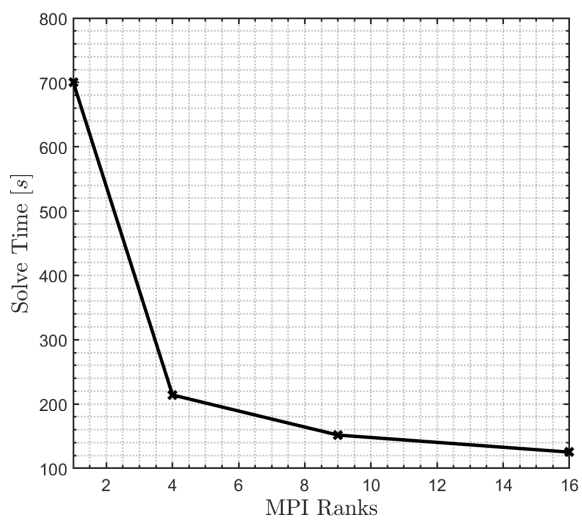
Figure A.3: Call trees showing inclusive times for profiling



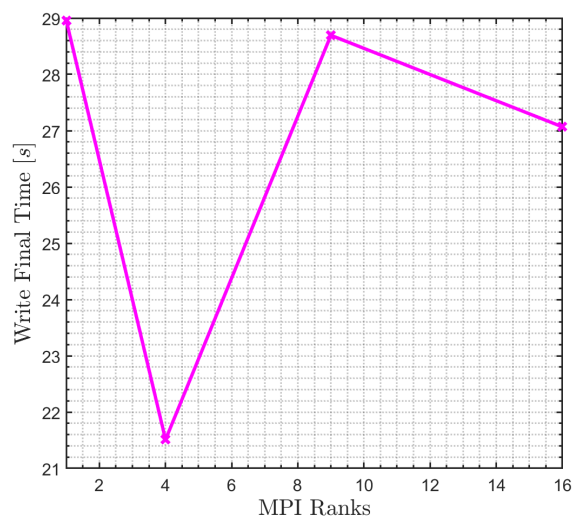
(a) Initialise



(b) Write Initial Conditions

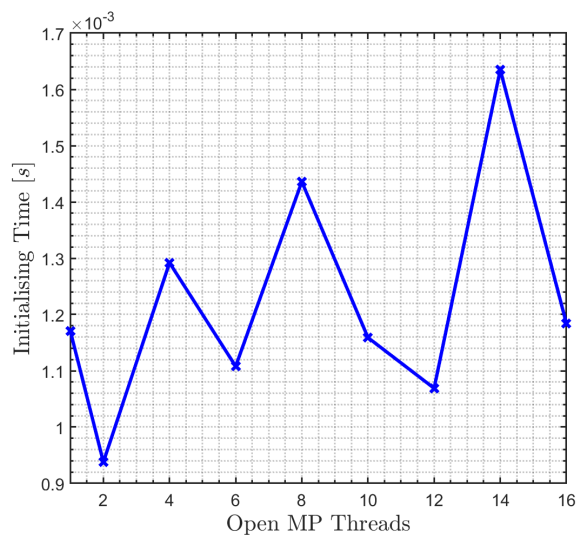


(c) Solve

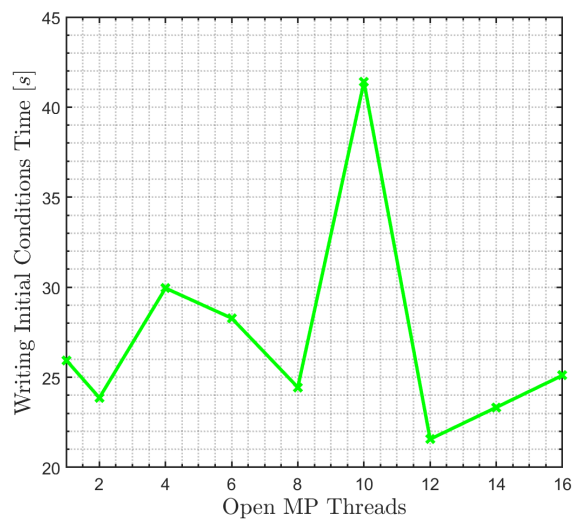


(d) Write Final Conditions

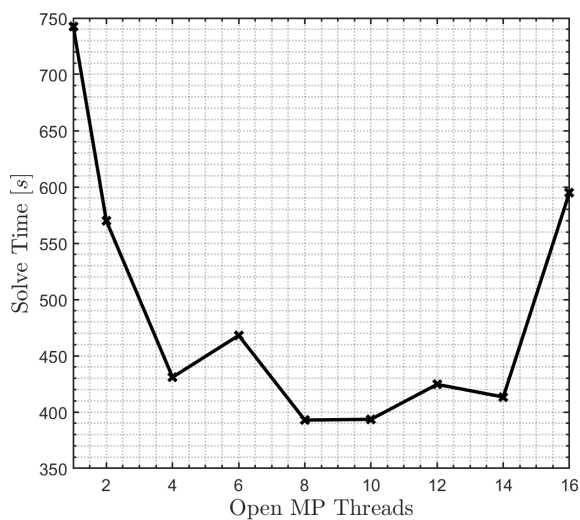
Figure A.4: Plots showing effect of MPI parallelisation on various functions for the “Large” settings



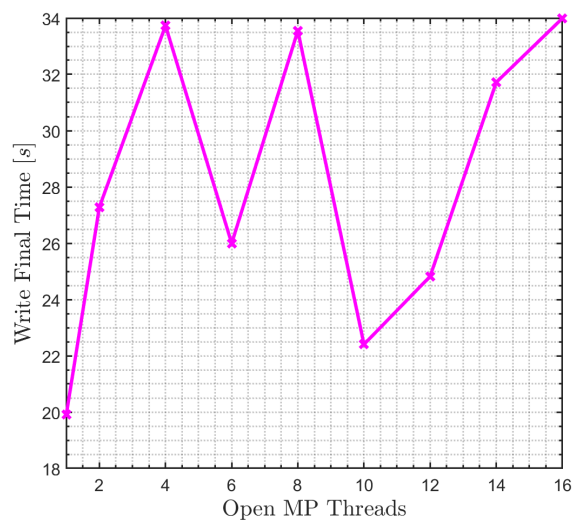
(a) Initialise



(b) Write Initial Conditions

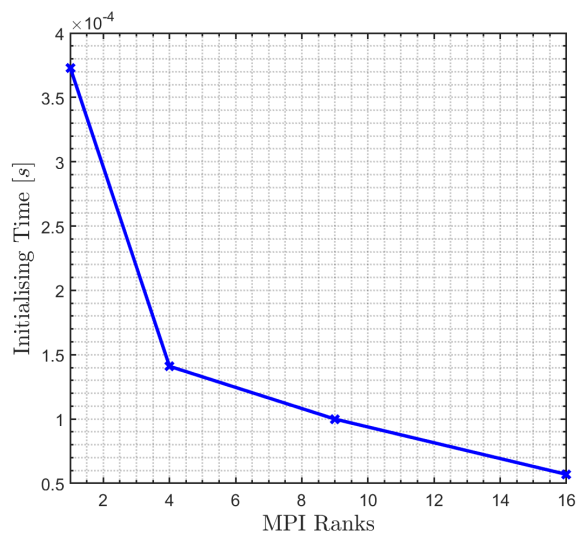


(c) Solve

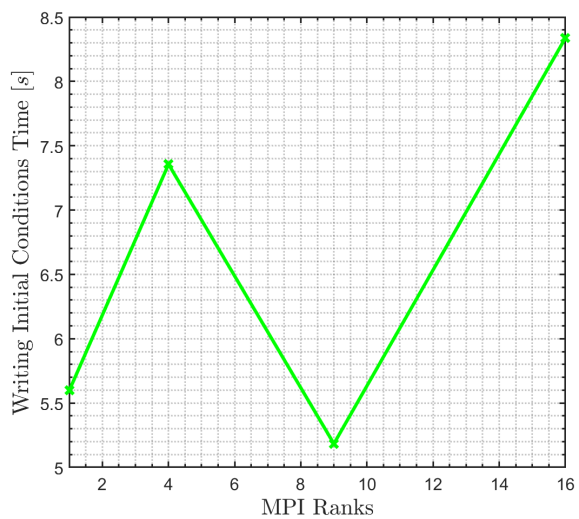


(d) Write Final Conditions

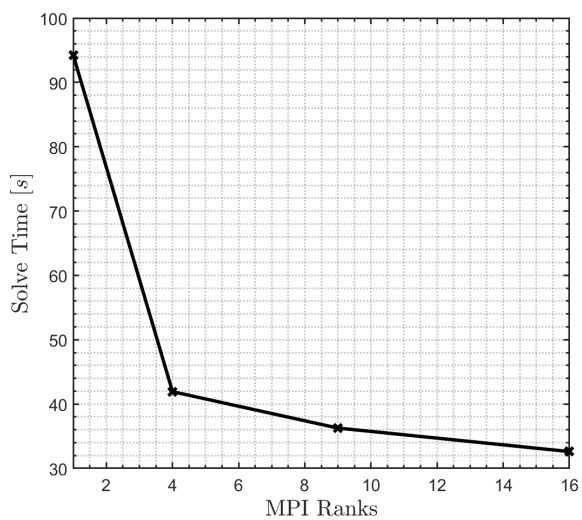
Figure A.5: Plots showing effect of Open MP parallelisation on various functions for the “Large” settings



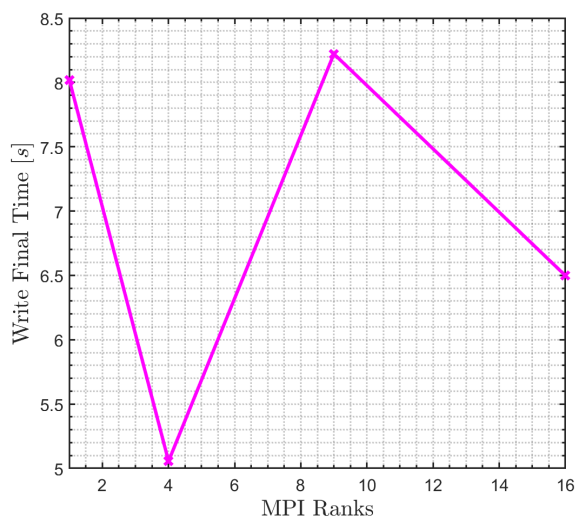
(a) Initialise



(b) Write Initial Conditions

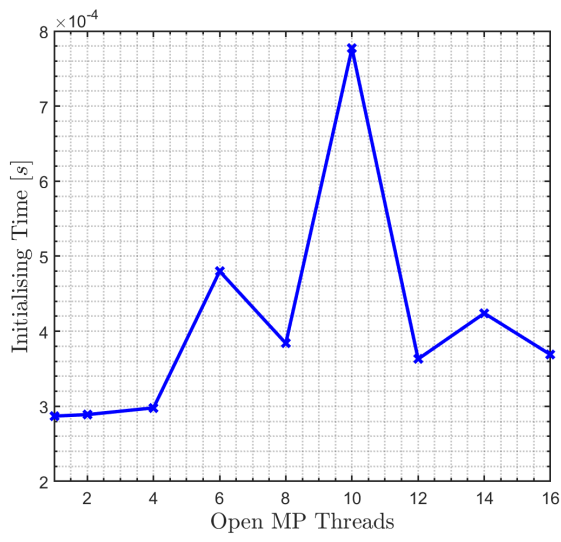


(c) Solve

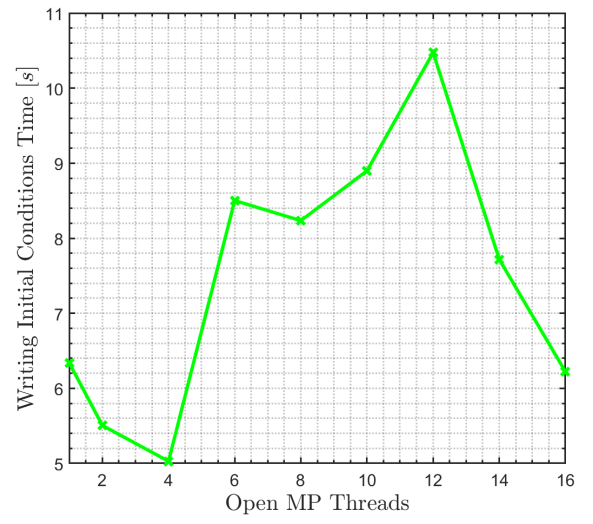


(d) Write Final Conditions

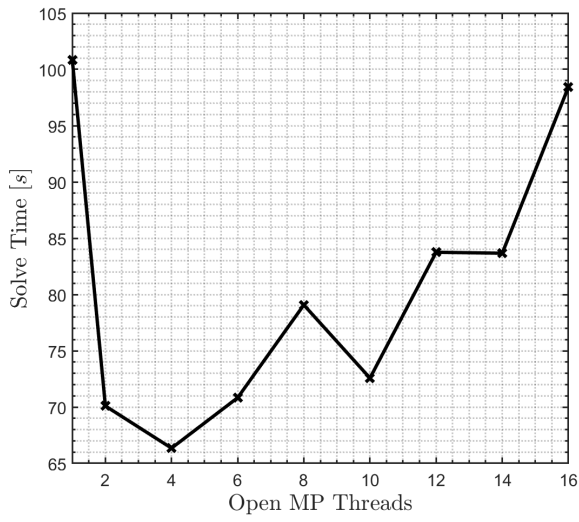
Figure A.6: Plots showing effect of MPI parallelisation on various functions for the “Small” settings



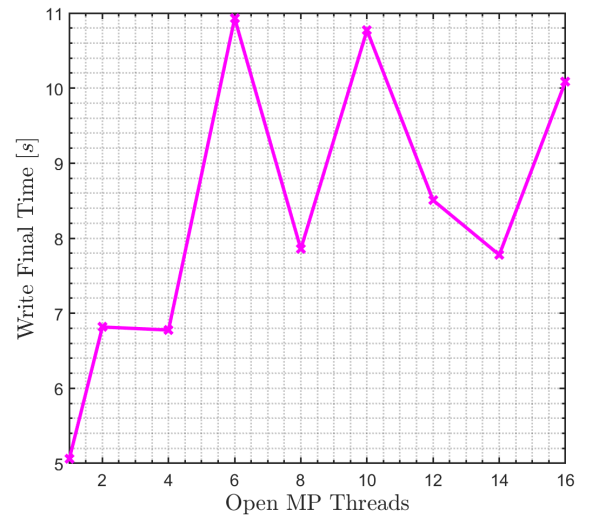
(a) Initialise



(b) Write Initial Conditions



(c) Solve



(d) Write Final Conditions

Figure A.7: Plots showing effect of Open MP parallelisation on various functions for the “Small” settings

Appendix B

Unit Test Code

```
1 #include "../include/MyTest.h"
2
3 // Main Test module to combine unit tests in
4 #define BOOST_TEST_MODULE Solver_and_Cavity_Test_Suite
5 #include <boost/test/included/unit_test.hpp>
6
7 // Apply predefined fixture for all tests and init MPI and OMP
8 BOOST_GLOBAL_FIXTURE(MPIFixture);
```

Code Snippet B.1: “UnitTest.cpp” that compiles all unit tests into one “out”

```
1 /**
2  * @file LDCPrintConfigTest.cpp
3  * @author Paul Kallarackel (paulkallarackel@gmail.com)
4  * @brief This file contains unit-test for print config fucntion on LDC class
5  * @version 0.1
6  * @date 2024-03-21
7  *
8  * @copyright Copyright (c) 2024
9  *
10 */
11 #include "../include/LidDrivenCavity.h"
12 #include "../include/MyTest.h"
13 #include <boost/test/unit_test.hpp>
14 #include <iostream>
15 #include <sstream>
16 #include <regex>
17 #include <cmath>
18 #include <fstream>
19 #include <iterator>
20
21
22 BOOST_AUTO_TEST_CASE(LidDrivenCavity_PRINTCONFIG_Test)
23 {
24     MPI_Barrier(MPI_COMM_WORLD); /// sync up test
25     /// determine local rank and global size
26     int world_size, world_rank;
27     MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);
28     MPI_Comm_size(MPI_COMM_WORLD, &world_size);
29     /// only need to check test on rank 0 as will be printed once
30     if (world_rank == 0){
31
32         std::cout << "\nLidDrivenCavity PrintConfig Test:" << std::endl;
33         /// Expected values for terminal print
34         double Lx = 1.0;
35         double Ly = 1.0;
36         double Nx = 9;
37         double Ny = 9;
38         double N = Nx * Ny;
39         double dt = 0.01;
40         double T = 1.0;
41         double Re = 10;
42         double Dx = Lx / (Nx - 1);
43         double Dy = Ly / (Ny - 1);
44         double steps = ceil(T / dt);
```

```

45
46 // storing expect val in vector and init computed vector
47 double *input = new double[11]{Nx, Ny, Dx, Dy, Lx, Ly, N, dt, steps, Re};
48 double *readOutput = new double[11]();
49
50 // Create and configure an instance of LidDrivenCavity
51 LidDrivenCavity *ldc = new LidDrivenCavity();
52 ldc->SetDomainSize(Lx, Ly);
53 ldc->SetGridSize(Nx, Ny);
54 ldc->SetTimeStep(dt);
55 ldc->SetFinalTime(T);
56 ldc->SetReynoldsNumber(Re);
57
58 // Redirect std::cout to a stringstream
59 std::stringstream buffer;
60 std::streambuf *prevCoutBuf = std::cout.rdbuf(buffer.rdbuf());
61
62 ldc->PrintConfiguration(); // Call the method to capture its output
63
64 std::cout.rdbuf(prevCoutBuf); // Restore the original buffer
65
66 std::string output = buffer.str();
67 // std::cout << "Output:\n" << output; // print captured output
68
69 // Regex to match a floating point number or integer
70 std::regex numberPattern("[+]?[0-9]*\\.?[0-9]+([eE][+]?[0-9]+)?");
71
72 // Use regex iterator to find all numbers in the output string and store in readOutput
73 std::sregex_iterator iter(output.begin(), output.end(), numberPattern);
74 std::sregex_iterator end;
75 int count = 0;
76 while (iter != end)
77 {
78     std::smatch match = *iter;
79     readOutput[count] = std::stod(match.str());
80     count++;
81     iter++;
82 }
83 // check 2-norm error between expected and printed
84 BOOST_CHECK(myTest::compareSolution(input, readOutput, 11, 1e-3));
85 // deallocate memory
86 delete ldc;
87 delete[] input;
88 delete[] readOutput;
89 }
90 // sync up ranks for future tests
91 MPI_Barrier(MPI_COMM_WORLD);
92 }

```

Code Snippet B.2: “LDCPrintConfigTest.cpp” that contains unit tests for LDC

```

1 /**
2  * @file SCGZeroTest.cpp
3  * @author Paul Kallarackel (paulkallarackel@gmail.com)
4  * @brief Contains unit test for zero input vorticity
5  * @version 0.1
6  * @date 2024-03-21
7  *
8  * @copyright Copyright (c) 2024
9  *
10 */
11 #include "../include/SolverCG.h"
12 #include "../include/LidDrivenCavity.h"
13 #include "../include/MyTest.h"
14 #include <boost/test/unit_test.hpp>

```

```

15 #include <iostream>
16 #include <cmath>
17
18
19 BOOST_AUTO_TEST_CASE(SolverCG_SOLVE_ZeroTest){
20     /// sync up ranks
21     MPI_Barrier(MPI_COMM_WORLD);
22     /// Initialise cart grid and rank info
23     int world_size, world_rank;
24     MPI_Comm cartComm;
25     MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);
26     MPI_Comm_size(MPI_COMM_WORLD, &world_size);
27     if (world_rank == 0) std::cout << "\nSolverCG Zero Test:" << std::endl;
28     int world_p = (int)sqrt(world_size);
29     const int dims = 2;
30     int sizes[dims] = {world_p, world_p};
31     int periods[dims] = {0, 0};
32     int reorder = 0;
33     MPI_Cart_create(MPI_COMM_WORLD, dims, sizes, periods, reorder, &cartComm);
34
35     int Nx = 1e2; // Example grid size
36     int Ny = 1e2; // Example grid size
37     double Lx = 1.0; // Example grid size
38     double Ly = 1.0; // Example grid size
39     double dx = Lx / (double)(Nx - 1);
40     double dy = Ly / (double)(Ny - 1);
41     LidDrivenCavity* ldc = new LidDrivenCavity();
42     ldc->SetDomainSize(Lx, Ly);
43     ldc->SetGridSize(Nx, Ny);
44     ldc->Initialise(world_p, world_rank, cartComm); /// set vorticity to 0
45
46     /// extract grid data fro local error in chunk
47     prl::gridData* GRID= ldc->getGRID();
48     SolverCG *solver = new SolverCG(Nx, Ny, dx, dy, GRID);
49     int *start = new int[2];
50     int *stop = new int[2];
51     GRID->getStart(start);
52     GRID->getStop(stop);
53     int Chunkx = GRID->getChunkx();
54     int Chunky = GRID->getChunky();
55     int size = (Chunkx+2)*(Chunky+2);
56     double *analytical = new double[size]();
57     double *solution = new double[size]();
58     double *locV = new double[size]();
59     solver->Solve(locV, solution, GRID);
60
61     /// calc and reduce local error
62     int idx;
63     double locError = 0.0;
64     double ERR = 0.0;
65     for (int i = start[0]; i <= stop[0]; ++i)
66     {
67         for (int j = start[1]; j <= stop[1]; ++j)
68         {
69             idx = LOCIDX(i-start[0], j-start[1]);
70             locError+= (analytical[idx]-solution[idx])*(analytical[idx]-solution[idx]);
71         }
72     }
73     MPI_Reduce(&locError, &ERR, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
74     if(world_rank==0){
75         ERR = sqrt(ERR);
76         BOOST_CHECK(myTest::compare(ERR, 0.001));
77     }
78
79     /// free up resources
80     delete solver;

```



```

81     delete ldc;
82     delete[] solution;
83     delete[] analytical;
84     delete[] locV;
85     delete[] start;
86     delete[] stop;
87     MPI_Comm_free(&cartComm);
88     MPI_Barrier(MPI_COMM_WORLD);
89 }

```

Code Snippet B.3: “SCGZeroTest.cpp” that contains unit tests for SCG

```

1  /**
2   * @file LDCWriteSolTest.cpp
3   * @author Paul Kallarackel (paulkallarackel@gmail.com)
4   * @brief Contains unit test for write solution on LDC class
5   * @version 0.1
6   * @date 2024-03-21
7   *
8   * @copyright Copyright (c) 2024
9   *
10  */
11 #include "../include/LidDrivenCavity.h"
12 #include "../include/MyTest.h"
13 #include <boost/test/unit_test.hpp>
14 #include <iostream>
15 #include <sstream>
16 #include <regex>
17 #include <vector>
18 #include <cmath>
19 #include <fstream>
20 #include <iterator>
21
22
23 BOOST_AUTO_TEST_CASE(LidDrivenCavity_WRITESOLUTION_Test)
24 {
25     /// sync up ranks for test
26     MPI_Barrier(MPI_COMM_WORLD);
27     /// initialis ecurrent rank, rank size, and cart grid
28     int world_size, world_rank;
29     MPI_Comm cartComm;
30     MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);
31     MPI_Comm_size(MPI_COMM_WORLD, &world_size);
32     if (world_rank == 0)
33         std::cout << "\nLidDrivenCavity WriteSolution Test:" << std::endl;
34     int world_p = (int)sqrt(world_size);
35     const int dims = 2;
36     int sizes[dims] = {world_p, world_p};
37     int periods[dims] = {0, 0};
38     int reorder = 0;
39     MPI_Cart_create(MPI_COMM_WORLD, dims, sizes, periods, reorder, &cartComm);
40     /// initlaise settings
41     double Lx = 1.0;
42     double Ly = 1.0;
43     double Nx = 9;
44     double Ny = 9;
45     double dt = 0.01;
46     double T = 1.0;
47     double Re = 10;
48     /// new LDC instance
49     LidDrivenCavity *ldc = new LidDrivenCavity();
50     ldc->SetDomainSize(Lx, Ly);
51     ldc->SetGridSize(Nx, Ny);
52     ldc->SetTimeStep(dt);
53     ldc->SetFinalTime(T);

```

```

54 ldc->SetReynoldsNumber(Re);
55 /// Initialise with zero IC (except U velocity on top wall) and write out to text
56 ldc->Initialise(world_p, world_rank, cartComm);
57 std::string filePath = "output/ic.txt";
58 ldc->WriteSolution(filePath);
59 /// Only need to check file data on one rank for simplicity
60 if (world_rank == 0)
61 {
62     /// Exoected values for test conditions: max number of empty lines, max number of values in a
        line, max number of data points
63     double *input = new double[3]{1, 6, Nx * Ny};
64     double *output = new double[3];
65
66     /// Open the file
67     std::ifstream inFile(filePath);
68     BOOST_REQUIRE(inFile.is_open()); /// Ensure the file is open
69
70     std::string line;
71     /// init counters
72     int linesWithValueCount = 0;
73     int emptyLineCount = 0;
74     int consecutiveEmptyLineCount = 0;
75     int maxNumberValues = 0;
76     int maxConsecutiveEmptyLineCount = 0;
77
78     /// reading in each line until eof
79     while (std::getline(inFile, line))
80     {
81         if (line.empty())
82         {
83             consecutiveEmptyLineCount++;
84             maxConsecutiveEmptyLineCount = max(consecutiveEmptyLineCount,
                maxConsecutiveEmptyLineCount);
85             emptyLineCount++;
86         }
87         else
88         {
89             /// Using stream iterator to extarct number of values
90             consecutiveEmptyLineCount = 0;
91             std::istringstream iss(line);
92             std::vector<double> values((std::istream_iterator<double>(iss)),
                std::istream_iterator<double>());
93             maxNumberValues = max((int)values.size(), maxNumberValues);
94             linesWithValueCount++;
95         }
96     }
97
98
99     output[0] = maxConsecutiveEmptyLineCount;
100     output[1] = maxNumberValues;
101     output[2] = linesWithValueCount;
102
103     BOOST_CHECK(myTest::compareSolution(input, output, 3, 1e-3));
104
105     /// free up memory
106     delete[] input;
107     delete[] output;
108 }
109 delete ldc;
110 MPI_Comm_free(&cartComm);
111 MPI_Barrier(MPI_COMM_WORLD);
112 }

```

Code Snippet B.4: "LDCWriteSolTest.cpp" that contains unit tests for LDC

```

2  * @file SCGAnalyticalTest.cpp
3  * @author Paul Kallarackel (paulkallarackel@gmail.com)
4  * @brief Contains unit-test for analytical sinusoidal function
5  * @version 0.1
6  * @date 2024-03-21
7  *
8  * @copyright Copyright (c) 2024
9  *
10 */
11 #include "../include/SolverCG.h"
12 #include "../include/LidDrivenCavity.h"
13 #include "../include/MyTest.h"
14 #include <boost/test/unit_test.hpp>
15 #include <iostream>
16 #include <cmath>
17
18
19
20 BOOST_AUTO_TEST_CASE(SolverCG_SOLVE_AnalyticalTest)
21 {
22     /// sync up test
23     MPI_Barrier(MPI_COMM_WORLD);
24     /// initialise grid, rank and size
25     int world_size, world_rank;
26     MPI_Comm cartComm;
27     MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);
28     MPI_Comm_size(MPI_COMM_WORLD, &world_size);
29     if (world_rank == 0) std::cout << "\nSolverCG Analytical Test:" << std::endl;
30     int world_p = (int)sqrt(world_size);
31     const int dims = 2;
32     int sizes[dims] = {world_p, world_p};
33     int periods[dims] = {0, 0};
34     int reorder = 0;
35     MPI_Cart_create(MPI_COMM_WORLD, dims, sizes, periods, reorder, &cartComm);
36
37     int Nx = 2e3; // Example grid size
38     int Ny = 2e3; // Example grid size
39     double Lx = 1.0; // Example grid size
40     double Ly = 1.0; // Example grid size
41     double dx = Lx / (double)(Nx - 1);
42     double dy = Ly / (double)(Ny - 1);
43     LidDrivenCavity* ldc = new LidDrivenCavity();
44     ldc->SetDomainSize(Lx, Ly);
45     ldc->SetGridSize(Nx, Ny);
46     ldc->Initialise(world_p, world_rank, cartComm);
47     prl::gridData* GRID= ldc->getGRID(); /// Extract grid data for local error calcs
48     SolverCG *solver = new SolverCG(Nx, Ny, dx, dy, GRID);
49     int *start = new int[2];
50     int *stop = new int[2];
51     GRID->getStart(start);
52     GRID->getStop(stop);
53     int Chunkx = GRID->getChunkx();
54     int Chunky = GRID->getChunky();
55     int size = (Chunkx+2)*(Chunky+2); /// size of local vectors
56     double *analytical = new double[size]();
57     double *solution = new double[size]();
58     double *locV = new double[size]();
59     const double k = 2.0 / Lx; // frequency to allow BC to be imposed
60     const double l = 2.0 / Ly;
61     int idx;
62     double x, y;
63     for (int i = start[0]; i <= stop[0]; ++i)
64     {
65         for (int j = start[1]; j <= stop[1]; ++j)
66         {
67             idx = LOCIDX(i-start[0], j-start[1]);

```

```

68         x = i*dx;
69         y = (Ny-1-j)*dy;
70         /// Analytical sinusoidal function
71         locV[idx]= M_PI * M_PI * (k * k + 1 * 1) * sin(M_PI * k * x) * sin(M_PI * l * y );
72         analytical[idx] = sin(M_PI * k * x) * sin(M_PI * l * y);
73     }
74 }
75 solver->Solve(locV,solution,GRID); // solve for analytical solution
76
77 /// locally calc error rand sum reduce
78 double locError = 0.0;
79 double ERR = 0.0;
80 for (int i = start[0]; i <= stop[0]; ++i)
81 {
82     for (int j = start[1]; j <= stop[1]; ++j)
83     {
84         idx = LOCIDX(i-start[0],j-start[1]);
85         locError+= (analytical[idx]-solution[idx])*(analytical[idx]-solution[idx]);
86     }
87 }
88 MPI_Reduce(&locError,&ERR,1,MPI_DOUBLE,MPI_SUM,0,MPI_COMM_WORLD);
89 ERR = sqrt(ERR);
90 if(world_rank==0)BOOST_CHECK(myTest::compare(ERR,0.001));
91
92 /// Free up resources
93 delete solver;
94 delete ldc;
95 delete[] solution;
96 delete[] analytical;
97 delete[] locV;
98 delete[] start;
99 delete[] stop;
100 MPI_Comm_free(&cartComm);
101 MPI_Barrier(MPI_COMM_WORLD);
102 }

```

Code Snippet B.5: “SCGAnalyticalTest.cpp” that contains unit tests for SCG

```

1 /**
2  * @file MyTest.cpp
3  * @author Paul Kallarackel (paulkallarackel@gmail.com)
4  * @brief Contains impementation of methods and test fixture in MyTest.h
5  * @version 0.1
6  * @date 2024-03-21
7  *
8  * @copyright Copyright (c) 2024
9  *
10 */
11 #include "../include/MyTest.h"
12 #include <cbblas.h>
13 #include <iostream>
14 #include <mpi.h>
15 #include <omp.h>
16 #include <cmath>
17 #include <boost/test/unit_test.hpp>
18
19 bool myTest::compareSolution(const double *numerical, const double *analytical, int size, double tol)
20 {
21     /// Uses CBLAS to calc 2norm error in vectors passed
22     double *ERR = new double[size];
23     cbblas_dcopy(size, analytical, 1, ERR, 1);
24     cbblas_daxpy(size, -1.0, numerical, 1, ERR, 1);
25     double e = cbblas_dnrm2(size, ERR, 1);
26     /// print error and if test pass
27     std::cout << "ERROR: " << e << std::endl;

```

```

28     if (e < tol)
29         std::cout << "Test Pass!" << std::endl;
30     delete[] ERR;
31     return e < tol;
32 }
33
34 bool myTest::compare(double e, double tol)
35 {
36     /// print error and if test pass
37     std::cout << "ERROR: " << e << std::endl;
38     if (e < tol)
39         std::cout << "Test Pass!" << std::endl;
40     return e < tol;
41 }
42
43 MPIFixture::MPIFixture()
44 {
45     /// Collects terminal arguments into main file
46     argc = boost::unit_test::framework::master_test_suite().argc;
47     argv = boost::unit_test::framework::master_test_suite().argv;
48
49     /// Initialising MPI
50     int err = MPI_Init(&argc, &argv);
51     if (err != MPI_SUCCESS)
52     {
53         msg = "ERROR initialising MPI";
54         BOOST_FAIL(msg);
55     }
56
57     /// Initialising Thread Number from env with serial fallback
58     int ThreadNum, ThreadID;
59     int DEFAULT_ThreadNum = 1;
60     char *env_var = getenv("OMP_NUM_THREADS");
61     if (env_var != NULL) {
62         ThreadNum = atoi(env_var); /// Convert string to integer
63     } else {
64         ThreadNum = DEFAULT_ThreadNum;
65         printf("OMP_NUM_THREADS is not set. Using default number of threads.\nSet env variable with \"
66             export OMP_NUM_THREADS=4\"\n");
67     }
68     omp_set_num_threads(std::min(ThreadNum, omp_get_max_threads()));
69
70     /// Get the rank and comm size on each process.
71     MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);
72     MPI_Comm_size(MPI_COMM_WORLD, &world_size);
73     if (world_size < 1)
74     {
75         msg = "ERROR use at least 1 process";
76         BOOST_FAIL(msg);
77     }
78     if (sqrt(world_size) != floor(sqrt(world_size)))
79     {
80         msg = "ERROR use square number process";
81         BOOST_FAIL(msg);
82     }
83     if (err != 0)
84     {
85         MPI_Finalize(); /// Ensure MPI is finalized if there's an error
86         BOOST_FAIL(msg);
87     }
88     if (world_rank == 0) std::cout << "Number of ranks: " << world_size << std::endl;
89     #pragma omp parallel private(ThreadID)
90     {
91         /// Obtain and print thread id
92         ThreadID = omp_get_thread_num();
93         /// Only master thread does this

```

```

93     if (ThreadID == 0 && world_rank == 0)
94     {
95         ThreadNum = omp_get_num_threads();
96         std::cout << "Number of threads = " << ThreadNum << std::endl;
97     }
98 }
99 // Sync up ranks before testing
100 MPI_Barrier(MPI_COMM_WORLD);
101
102 }
103
104 MPIFixture::~MPIFixture()
105 {
106     // Close out MPI once all unit tests complete
107     MPI_Finalize();
108 }

```

Code Snippet B.6: “MyTest.cpp” that contains Boost global fixture definition

Appendix C

Parallel Functions

```
1  /**
2  * @file ParallelFunc.h
3  * @author Paul Kallarackel (paulkallarackel@gmail.com)
4  * @brief Function details main MPI Conceptual grid implemntation
5  * Contains logic to distribute Nx*Ny grid amongst ranks and pass on global position data like coords
   and neighbours
6  * @version 0.1
7  * @date 2024-03-20
8  *
9  * @copyright Copyright (c) 2024
10 *
11 */
12 #pragma once
13 #include <mpi.h>
14 #include <string>
15 #include <unistd.h>
16
17 #define EXCHGIDX(I, J) ((J) * (Chunkx + 2) + (I)) ///< Defines Row access logic of local vector
   including padding
18 #define LOCIDX(I, J) EXCHGIDX(I + 1, J + 1) ///< allows for excluding padding in calcs
19 /**
20 * @brief Holds logic to distribute Nx*Ny grid and rank-to-rank communication
21 *
22 */
23 namespace prl
24 {
25
26     /**
27     * @brief Local and Global data object with rank-to-rank comms
28     *
29     */
30     struct gridData
31     {
32
33     public:
34         int offsetX; ///< variable used to translate local vector coords to global grid position in x
35         int offsetY; ///< variable used to translate local vector coords to global grid position in y
36         MPI_Datatype x_edge_type; ///< Custom MPI datatype to send edge row data to other ranks
37         MPI_Datatype y_edge_type; ///< Custom MPI datatype to send edge col data to other ranks
38
39         /**
40         * @brief Custom construct a new grid Data object
41         *
42         * @param init_Nx Number of x points in LidDrivenCavity object
43         * @param init_Ny Number of y points in LidDrivenCavity object
44         * @param init_world_p Number of ranks per dimension in LidDrivenCavity object
45         * @param init_world_rank Current rank number in LidDrivenCavity object
46         * @param init_cartComm Current Cartesian MPI communicator in LidDrivenCavity object
47         */
48         gridData(int init_Nx, int init_Ny, int init_world_p, int init_world_rank, MPI_Comm &
            init_cartComm);
49
50         /**
51         * @brief Default construct a new grid Data object
52         *
53         */
54     }
```



```

54 gridData();
55
56 /**
57  * @brief Destroy the grid Data object
58  *
59  */
60 ~gridData();
61
62 /**
63  * @defgroup getCoord Get Cartesian Coordinates
64  * @param[in,out] COORD Pointer to an array to store the Cartesian coordinates.
65  * @return
66  * @{
67  *
68  */
69 /**
70  * @note coords will be in domain [0,p)
71  * @brief Get the Cartesian coordinates of rank.
72  */
73 void getCartCoord(int *COORD);
74 /**
75  * @note Coords will be in global Nx*Ny domain
76  * @brief Get the top-left coord of the local vector
77  */
78 void getStart(int *COORD);
79 /**
80  * @note Coords will be in global Nx*Ny domain
81  * @brief Get the bottom-right coord of the local vector
82  */
83 void getStop(int *COORD);
84 /**@}*/
85
86
87 /**
88  * @defgroup getChunkSize Get Chunk Size
89  * @note chunk is the size of local vector once global LDC cavity has been divided into ranks
90  * @return length of vector in given direction.
91  * @{
92  *
93  */
94 int getChunkx(); ///< Get the size of the chunk in the x-direction.
95 int getChunky(); ///< Get the size of the chunk in the y-direction.
96 /**@}*/
97
98 /**
99  * @defgroup getCartNeighbour Get Cartesian Neighbours
100  * @return Rank of the rank of process within cart comm.
101  * @note If no neighbour available, returns -2
102  * @{
103  *
104  */
105 int getLeft(); ///< Get left neighbour
106 int getRight(); ///< Get right neighbour
107 int getUp(); ///< Get up neighbour
108 int getDown(); ///< Get down neighbour
109 /// @note will be same as world rank in most cases but implemented in case MPI distributes
    differently
110 int getCenter(); ///< Get current rank within cartesian comm
111 /**@}*/
112
113
114 /**
115  * @brief Get the rank of the current process in the cart communicator.
116  *
117  * @return Rank of the current process.
118  */

```

```

119 int getRank();
120
121 /**
122  * @defgroup convLocGlo Convert local <--> global coords
123  * @note converts vectors from local [Chunkx,Chunky] domain to and from global [Nx,Ny] domain
124  * @param[in,out] locCoord Pointer to local coordiante array
125  * @param[in,out] gloCoord Pointer to local coordiante array
126  * @return
127  * @{
128  *
129  */
130 void loc2glo(int *locCoord, int *gloCoord); ///< converts local vector coords into global
131 void glo2loc(int *locCoord, int *gloCoord); ///< Converts global vector coords into local
132 /**@}*/
133
134 /**
135  * @brief Main function used to eexchange any local vector with neighbour ranks and receive
136  *        into padding
137  * Uses Non blocking MPI_Isend and MPI_Irecv to send to neighbour ranks
138  * @param[in,out] data Any choice of local vector such as LidDrivenCavity::v
139  * @warning cartesian grid must have been initliased first, with new LidDrivenCavity instance
140  *        and calling LidDrivenCavity::Initialise()
141  */
142 void exchangeGhost(double *data);
143
144 /**
145  * @brief Zeros out padding (edge data) for use in 2-norm error calcs in testing
146  * Goes around the padding of the vectors and zeros
147  * @param[in,out] data
148  */
149 void edgeZero(double *data);
150
151 private:
152 int Nx; ///< Number of global x points
153 int Ny; ///< Number of global y points
154 int remx; ///< Remainder of global x length of grid by number of ranks per dimension
155 int remy; ///< Remainder of global y length of grid by number of ranks per dimension
156 int Chunkx; ///< Local size of vector in x dimension
157 int Chunky; ///< Local size of vector in y dimension
158 int world_rank; ///< Current global Rank number
159 int cartRank; ///< Current Rank number in created grid
160 int world_p; ///< Number of ranks per dimension
161 int world_size; ///< Total number of ranks
162 int *cartCoord; ///< Coordinates of current rank in cartesian grid
163 int *start; ///< Top left corner coordiantes of local vector in global form
164 int *stop; ///< Bottom right corner coordiantes of local vector in global form
165 int leftNeighbour; ///< Rank to the left
166 int upNeighbour; ///< Rank above
167 int downNeighbour; ///< Rank below
168 int rightNeighbour; ///< Rank to the right
169 MPI_Comm cartComm; ///< Cartesian Communicator for rank grid
170
171 /**
172  * @brief updates offset variable for local->global coordiantes
173  *
174  */
175 void updateOffset();
176
177 /**
178  * @brief Calculates offset variable for local->global coordiantes
179  * @param N Number of points in dimension
180  * @param p Number of ranks in dimension
181  * @param rank Current position of rank along dimension
182  * @return
183  */

```

```

183     */
184     int calcOffset(int N, int p, int rank);
185
186     /**
187      * @brief Finds min chunk size for ranks along dimension
188      *
189      * @param N Number of points in dimension
190      * @param p Number of ranks in dimension
191      * @return int
192      */
193     int minChunkSize(int N, int p);
194
195     /**
196      * @brief returns remainder of points divided by ranks in dimension
197      *
198      * @param N Number of points in dimension
199      * @param p Number of ranks in dimension
200      * @return int
201      */
202     int rem(int N, int p);
203
204     /**
205      * @brief Calculates and applies grid data to object
206      *
207      */
208     void init();
209 };
210
211 /**
212  * @brief Debugging statemnt when working with mpi that gives rank timestamp as improvement over
213  * normal print
214  *
215  * @param rank Current Rank number
216  * @param format String used to print
217  * @param ... variable arguements like Printf()
218  * @note Will print timestamp|rank|print statement
219  * @note use this for debugging any mpi code
220  */
221 void debug(int rank, const char *format, ...);
222
223 /**
224  * @brief Get Current time
225  *
226  * @return double
227  * @note creates a timer object used for debuggin mpi at beginning of program
228  */
229 double get_timer();
230
231 /**
232  * @brief Prints out matrix stored in row major
233  *
234  * @param m Number of rows
235  * @param n Number of cols
236  * @param H Matrix pointer
237  * @warning matrix must be stored in row major for print statemnt to work
238  */
239 void PrintRowMatrix(int m, int n, double *H);
240
241 /**
242  * @brief Copy of sleep function for debugging with MPI
243  *
244  * @param n Number of seconds to wait
245  */
246 void time(int n);

```

```

1 #include "../include/ParallelFunc.h"
2 #include <iomanip>
3 #include <mpi.h>
4 #include <chrono>
5 #include <unistd.h>
6 #include <string>
7 #include <omp.h>
8
9 prl::gridData::gridData(int init_Nx, int init_Ny, int init_world_p, int init_world_rank, MPI_Comm &
   init_cartComm)
10 {
11     /// Assign global LDC parameters to each ranks GRID object
12     Nx = init_Nx;
13     Ny = init_Ny;
14     world_rank = init_world_rank;
15     world_p = init_world_p;
16     cartComm = init_cartComm;
17     /// Initialise local vectors based on these parameters
18     init();
19 }
20
21 prl::gridData::gridData()
22 {
23 }
24
25 void prl::gridData::init()
26 {
27     world_size = world_p * world_p;
28     /// Reset Offset
29     offsetX = 0;
30     offsetY = 0;
31
32     /// get ranks coordinates within cartesian communicator
33     cartCoord = new int[2];
34     MPI_Cart_coords(cartComm, world_rank, 2, cartCoord);
35
36     /// calc remainder for chunk size calc
37     remx = rem(Nx, world_p);
38     remy = rem(Ny, world_p);
39
40     /// Calc chunk size based of ranks coordinates in grid
41     int minx = minChunkSize(Nx, world_p);
42     Chunkx = cartCoord[1] < remx ? minx + 1 : minx;
43     int miny = minChunkSize(Ny, world_p);
44     Chunky = cartCoord[0] < remy ? miny + 1 : miny;
45
46     /// find top left and bottom right global coord for ranks local vector
47     start = new int[2]();
48     stop = new int[2]{Chunkx - 1, Chunky - 1};
49     loc2glo(start, start);
50     loc2glo(stop, stop);
51
52     /// Find each neighbour of rank: -2 if no neighbour
53     MPI_Cart_rank(cartComm, cartCoord, &cartRank);
54     MPI_Cart_shift(cartComm, 0, 1, &upNeighbour, &downNeighbour);
55     MPI_Cart_shift(cartComm, 1, 1, &leftNeighbour, &rightNeighbour);
56
57     /// X edge to send
58     MPI_Type_vector(Chunkx, 1, 1, MPI_DOUBLE, &x_edge_type);
59     MPI_Type_commit(&x_edge_type);
60

```

```

61     /// Y edge to send
62     MPI_Type_vector(Chunky, 1, (Chunkx + 2), MPI_DOUBLE, &y_edge_type);
63     MPI_Type_commit(&y_edge_type);
64 }
65
66 prl::gridData::~gridData()
67 {
68     /// delete dynamic error allocated
69     delete[] cartCoord;
70     cartCoord = nullptr;
71     delete[] start;
72     start = nullptr;
73     delete[] stop;
74     stop = nullptr;
75     MPI_Type_free(&x_edge_type);
76     MPI_Type_free(&y_edge_type);
77 }
78
79 int prl::gridData::getChunkx()
80 {
81     return Chunkx;
82 }
83
84 int prl::gridData::getChunky()
85 {
86     return Chunky;
87 }
88
89 int prl::gridData::getDown()
90 {
91     return downNeighbour;
92 }
93
94 int prl::gridData::getCenter()
95 {
96     return cartRank;
97 }
98
99 int prl::gridData::getUp()
100 {
101     return upNeighbour;
102 }
103
104 int prl::gridData::getLeft()
105 {
106     return leftNeighbour;
107 }
108
109 int prl::gridData::getRight()
110 {
111     return rightNeighbour;
112 }
113
114 void prl::gridData::getCartCoord(int *COORD)
115 {
116     COORD[0] = cartCoord[0];
117     COORD[1] = cartCoord[1];
118 }
119
120 void prl::gridData::getStart(int *COORD)
121 {
122     COORD[0] = start[0];
123     COORD[1] = start[1];
124 }
125
126 void prl::gridData::getStop(int *COORD)

```

```

127 {
128     COORD[0] = stop[0];
129     COORD[1] = stop[1];
130 }
131
132 void prl::gridData::loc2glo(int *locCoord, int *gloCoord)
133 {
134     updateOffset();
135     /// global form of local coords is offset base dof where rank is in virt top.
136     gloCoord[0] = locCoord[0] + offsetX;
137     gloCoord[1] = locCoord[1] + offsetY;
138 }
139
140 void prl::gridData::glo2loc(int *locCoord, int *gloCoord)
141 {
142     updateOffset();
143     /// global form of local coords is offset base dof where rank is in virt top.
144     locCoord[0] = gloCoord[0] - offsetX;
145     locCoord[1] = gloCoord[1] - offsetY;
146 }
147
148 void prl::gridData::updateOffset()
149 {
150     /// X: used cartCoord[1] as (y,x) in MPI
151     offsetX = calcOffset(Nx, world_p, cartCoord[1]);
152     /// Y: used cartCoord[0] as (y,x) in MPI
153     offsetY = calcOffset(Ny, world_p, cartCoord[0]);
154 }
155
156 int prl::gridData::rem(int N, int p)
157 {
158     return N % p;
159 }
160
161 int prl::gridData::calcOffset(int N, int p, int rank)
162 {
163     /// Finds offset based of whether ``big'' or ``small'' Chunk
164     int r = rem(N, p);
165     int min = (N - r) / p;
166     /// Big
167     if (rank < r)
168     {
169         return rank * (min + 1);
170     }
171     /// small
172     else
173     {
174         return r * (min + 1) + (rank - r) * min;
175     }
176 }
177
178
179 int prl::gridData::minChunkSize(int N, int p)
180 {
181     /// Min chunk size from load balncing algo
182     int r = rem(N, p);
183     return (N - r) / p;
184 }
185
186 double prl::get_timer()
187 {
188     using namespace std::chrono;
189     static auto start = high_resolution_clock::now(); /// Store the start time at first call
190     auto now = high_resolution_clock::now(); /// Current time at each call
191     auto elapsed = duration_cast<nanoseconds>(now - start).count(); /// Elapsed time in nanoseconds
192     return static_cast<double>(elapsed) / 1e9; /// Convert to double and return

```

```

193 }
194
195 void prl::debug(int rank, const char *format, ...)
196 {
197     /// Uses variabel args to maintain printf functionality with timestamp and rnak number
198     va_list args;
199     va_start(args, format);
200     printf("%12.6f|%2d|", get_timer(), rank);
201     vprintf(format, args);
202     va_end(args);
203 }
204
205 void prl::gridData::exchangeGhost(double *data)
206 {
207     /// Number of rank neighbours
208     int nbrs = (int)(upNeighbour >= 0) + (int)(downNeighbour >= 0) + (int)(leftNeighbour >= 0) + (int)
        (rightNeighbour >= 0);
209
210     /// Number of status and requests
211     MPI_Status recStat[nbrs];
212     MPI_Request recReq[nbrs];
213     MPI_Request sendReq[nbrs];
214
215     /// If Neighbour found iterate count such that request array indexed properly as message sent
216     int count = 0;
217
218     /// Checks if up neighbour
219     if (upNeighbour >= 0)
220     {
221         /// Non blocking REceive into padding
222         MPI_Isend(&data[EXCHGIDX(1, 1)], 1, x_edge_type, upNeighbour, 1, cartComm, &sendReq[count]);
223         /// If terminal print requested
224         /// Non blocking REceive into padding
225         MPI_Irecv(&data[EXCHGIDX(1, 0)], 1, x_edge_type, upNeighbour, 2, cartComm, &recReq[count]);
226         count++;
227     }
228     if (downNeighbour >= 0)
229     {
230         /// Non blocking REceive into padding
231         MPI_Irecv(&data[EXCHGIDX(1, Chunky + 1)], 1, x_edge_type, downNeighbour, 1, cartComm, &recReq[
            count]);
232
233         /// Non blocking REceive into padding
234         MPI_Isend(&data[EXCHGIDX(1, Chunky)], 1, x_edge_type, downNeighbour, 2, cartComm, &sendReq[
            count]);
235
236         count++;
237     }
238     if (leftNeighbour >= 0)
239     {
240         /// Non blocking REceive into padding
241         MPI_Isend(&data[EXCHGIDX(1, 1)], 1, y_edge_type, leftNeighbour, 3, cartComm, &sendReq[count]);
242         /// Non blocking REceive into padding
243         MPI_Irecv(&data[EXCHGIDX(0, 1)], 1, y_edge_type, leftNeighbour, 4, cartComm, &recReq[count]);
244         count++;
245     }
246     if (rightNeighbour >= 0)
247     {
248         /// Non blocking REceive into padding
249         MPI_Irecv(&data[EXCHGIDX(Chunkx + 1, 1)], 1, y_edge_type, rightNeighbour, 3, cartComm, &recReq[
            count]);
250         /// Non blocking REceive into padding
251         MPI_Isend(&data[EXCHGIDX(Chunkx, 1)], 1, y_edge_type, rightNeighbour, 4, cartComm, &sendReq[
            count]);
252         count++;
253     }

```

```

254 // Free the MPI requests
255 if (count > 0)
256 {
257     for (int i = 0; i < count; ++i)
258     {
259         if(sendReq[i] != MPI_REQUEST_NULL) MPI_Request_free(&sendReq[i]);
260     }
261 }
262 /// Wait on Receives as data needed for next operation
263 MPI_Waitall(count, recReq, recStat);
264 }
265
266 void prl::gridData::edgeZero(double *data)
267 {
268     /// Go around edges of vector and zero out
269     #pragma omp parallel for schedule(static)
270     for (int i = 0; i < Chunkx + 2; ++i)
271     {
272         data[EXCHGIDX(i, 0)] = 0.0;
273     }
274     #pragma omp parallel for schedule(static)
275     for (int i = 0; i < Chunkx + 2; ++i)
276     {
277         data[EXCHGIDX(i, Chunky + 1)] = 0.0;
278     }
279     #pragma omp parallel for schedule(static)
280     for (int j = 0; j < Chunky + 2; ++j)
281     {
282         data[EXCHGIDX(0, j)] = 0.0;
283     }
284     #pragma omp parallel for schedule(static)
285     for (int j = 0; j < Chunky + 2; ++j)
286     {
287         data[EXCHGIDX(Chunkx + 1, j)] = 0.0;
288     }
289 }
290
291 void prl::PrintRowMatrix(int m, int n, double *H)
292 {
293     for (int i = 0; i < n; ++i)
294     {
295         std::cout << "Row " << i << " : ";
296         for (int j = 0; j < m; ++j)
297         {
298             std::cout << std::setw(5) << H[i * m + j] << " ";
299         }
300         std::cout << std::endl;
301     }
302     std::cout << std::endl;
303 }
304
305 void prl::time(int n)
306 {
307     sleep(n);
308 }

```

Code Snippet C.2: “ParallelFunc.cpp” that implements the “dataGrid” object and neighbour-rank exchange methods

Appendix D

Lid Driven Cavity

```
1  /**
2   * @file LidDrivenCavity.cpp
3   * @author Paul Kallarackel
4   * @brief This file implements the methods found in LidDrivenCavity.h
5   * @version 0.1
6   * @date 2024-02-28
7   *
8   * @copyright Copyright (c) 2024
9   *
10  */
11 #include <iostream>
12 #include <iomanip>
13 #include <fstream>
14 #include <cstring>
15 #include <cmath>
16 #include "../include/ParallelFunc.h"
17 #include <mpi.h>
18
19 using namespace std;
20 #include <sstream>
21
22 #include <cbblas.h>
23 #include "../include/LidDrivenCavity.h"
24 #include "../include/SolverCG.h"
25
26 LidDrivenCavity::LidDrivenCavity()
27 {
28 }
29 LidDrivenCavity::~LidDrivenCavity()
30 {
31     CleanUp();
32 }
33 prl::gridData *LidDrivenCavity::getGRID()
34 {
35     return GRID;
36 }
37 void LidDrivenCavity::SetDomainSize(double xlen, double ylen)
38 {
39     this->Lx = xlen;
40     this->Ly = ylen;
41     UpdateDxDy();
42 }
43 void LidDrivenCavity::SetGridSize(int nx, int ny)
44 {
45     this->Nx = nx;
46     this->Ny = ny;
47     UpdateDxDy();
48 }
49 void LidDrivenCavity::SetTimeStep(double deltat)
50 {
51     this->dt = deltat;
52 }
53 void LidDrivenCavity::SetFinalTime(double finalt)
54 {
55     this->T = finalt;
56 }
```

```

57 void LidDrivenCavity::SetReynoldsNumber(double re)
58 {
59     this->Re = re;
60     this->nu = 1.0 / re;
61 }
62 void LidDrivenCavity::Initialise(int p, int rank, MPI_Comm &comm)
63 {
64     CleanUp();
65     /// Init vorticity, stream function and temporary vector for simulation
66     CartInit(p, rank, comm);
67     /// Init new solver with domain size and discretisation
68     cg = new SolverCG(Nx, Ny, dx, dy, GRID);
69 }
70 void LidDrivenCavity::Integrate()
71 {
72     /// number of timesteps
73     int NSteps = ceil(T / dt);
74     for (int t = 0; t < NSteps; ++t)
75     {
76         if (GRID->getCenter() == 0)
77             std::cout << "Step: " << setw(8) << t
78                 << " Time: " << setw(8) << t * dt
79                 << std::endl;
80         Advance();
81     }
82 }
83 void LidDrivenCavity::WriteSolution(std::string file)
84 {
85     int *start = new int[2];
86     int *stop = new int[2];
87     int *currCoord = new int[2];
88     int Chunkx = GRID->getChunkx();
89     int Chunky = GRID->getChunky();
90     GRID->getStart(start);
91     GRID->getStop(stop);
92     bool wallTop = GRID->getUp() < 0;
93     bool wallBottom = GRID->getDown() < 0;
94     bool wallLeft = GRID->getLeft() < 0;
95     bool wallRight = GRID->getRight() < 0;
96     int size = (Chunkx + 2) * (Chunky + 2);
97     double *u0 = new double[size]();
98     double *u1 = new double[size]();
99     GRID->exchangeGhost(s);
100     #pragma omp parallel for schedule(static)
101     for (int j = (!wallTop ? 0 : 1); j < (!wallBottom ? Chunky : Chunky - 1); ++j)
102     {
103         for (int i = (!wallLeft ? 0 : 1); i < (!wallRight ? Chunkx : Chunkx - 1); ++i)
104         {
105
106             /// x velocity:  $u = d(\psi)/dy$ 
107             u0[LOCIDX(i, j)] = (s[LOCIDX(i, j - 1)] - s[LOCIDX(i, j)]) / dy;
108             /// y velocity:  $-v = d(\psi)/dx$ 
109             u1[LOCIDX(i, j)] = -(s[LOCIDX(i + 1, j)] - s[LOCIDX(i, j)]) / dx;
110         }
111     }
112     if (wallTop)
113     {
114         #pragma omp parallel for schedule(static)
115         for (int i = 0; i < Chunkx; ++i)
116         {
117             /// freestream velocity
118             u0[LOCIDX(i, 0)] = U;
119         }
120     }
121     int k = 0;
122     double x, y;

```

```

123 std::ostringstream oss;
124 oss << file << " ";
125 file = oss.str();
126 if (GRID->getCenter() == 0)
127 {
128     std::cout << "Writing file " << file << std::endl;
129     std::ofstream f(file.c_str());
130     f.close();
131 }
132 std::stringstream ss; // stringstream to collect data
133 /**
134  * synchronously iterate thorough global grid
135  * and check if coord exists in rank, if so print it to maintain order.
136  *
137  */
138 for (int i = 0; i < Nx; ++i)
139 {
140     for (int j = Ny - 1; j >= 0; --j)
141     {
142         MPI_Barrier(MPI_COMM_WORLD);
143         if (start[0] <= i && stop[0] >= i && start[1] <= j && stop[1] >= j)
144         {
145             std::ofstream f(file.c_str(), std::ios::app);
146             ss.str(""); // Clear the contents of the stringstream
147             ss.clear(); // Reset any error flags
148             currCoord[0] = i;
149             currCoord[1] = j;
150             GRID->glo2loc(currCoord, currCoord);
151             // array index of coordinates
152             k = LOCIDX(currCoord[0], currCoord[1]);
153             x = i * dx;
154             y = (Ny - 1 - j) * dy;
155             // Collecting data into stringstream
156             ss << x << " " << y << " " << v[k] << " " << s[k] << " " << u0[k] << " " << u1[k] << std
157                 ::endl;
158             f << ss.str();
159             f.close();
160         }
161     }
162     if (GRID->getCenter() == 0)
163     {
164         std::ofstream f(file.c_str(), std::ios::app);
165         f << std::endl; // Add a newline after each row for clarity
166         f.close();
167     }
168     delete[] u0;
169     delete[] u1;
170     delete[] start;
171     delete[] stop;
172     delete[] currCoord;
173 }
174 void LidDrivenCavity::PrintConfiguration()
175 {
176     cout << "Grid size: " << Nx << " x " << Ny << endl;
177     cout << "Spacing: " << dx << " x " << dy << endl;
178     cout << "Length: " << Lx << " x " << Ly << endl;
179     cout << "Grid pts: " << Npts << endl;
180     cout << "Timestep: " << dt << endl;
181     cout << "Steps: " << ceil(T / dt) << endl;
182     cout << "Reynolds number: " << Re << endl;
183     cout << "Linear solver: preconditioned conjugate gradient" << endl;
184     cout << endl;
185     if (nu * dt / dx / dy > 0.25)
186     {
187         cout << "ERROR: Time-step restriction not satisfied!" << endl;

```

```

188     cout << "Maximum time-step is " << 0.25 * dx * dy / nu << endl;
189     exit(-1);
190 }
191 }
192
193 void LidDrivenCavity::UpdateDxDy()
194 {
195     /// calculating grid size and number of poitns
196     dx = Lx / (Nx - 1);
197     dy = Ly / (Ny - 1);
198     Npts = Nx * Ny;
199 }
200
201 void LidDrivenCavity::Advance()
202 {
203     /// divisors to be used and multiplied by to optimise code
204     double dxi = 1.0 / dx;
205     double dyi = 1.0 / dy;
206     double dx2i = 1.0 / dx / dx;
207     double dy2i = 1.0 / dy / dy;
208     int Chunkx = GRID->getChunkx();
209     int Chunky = GRID->getChunky();
210     GRID->exchangeGhost(s);
211     bool wallTop = GRID->getUp()<0;
212     bool wallBottom = GRID->getDown()<0;
213     bool wallLeft = GRID->getLeft()<0;
214     bool wallRight = GRID->getRight()<0;
215     int startY = (!wallTop ? 0 : 1);
216     int startX = (!wallLeft ? 0 : 1);
217     int stopY = (!wallBottom ? Chunky : Chunky - 1);
218     int stopX = (!wallRight ? Chunkx : Chunkx - 1);
219     int N = stopX - startX;
220     double* tmp1ROW= nullptr;
221     double* tmp2ROW= nullptr;
222     double* tmp3ROW= nullptr;
223     double* tmp4ROW= nullptr;
224     double* tmp5ROW= nullptr;
225     double* tmp6ROW= nullptr;
226     double* tmp7ROW= nullptr;
227     /// Boundary node vorticity - Obtained using equations 6-9 on "brief.pdf"
228     if (wallTop)
229     {
230         #pragma omp parallel for schedule(static)
231         for (int i = startX; i < stopX; ++i)
232         {
233             v[LOCIDX(i, 0)] = 2.0 * dy2i * (s[LOCIDX(i, 0)] - s[LOCIDX(i, 1)]) - 2.0 * dyi * U;
234         }
235     }
236     if (wallBottom)
237     {
238         #pragma omp parallel for schedule(static)
239         for (int i = startX; i < stopX; ++i)
240         {
241             v[LOCIDX(i, Chunky - 1)] = 2.0 * dy2i * (s[LOCIDX(i, Chunky - 1)] - s[LOCIDX(i, Chunky - 2)]);
242         }
243     }
244     if (wallLeft)
245     {
246         #pragma omp parallel for schedule(static)
247         for (int j = startY; j < stopY; ++j)
248         {
249             v[LOCIDX(0, j)] = 2.0 * dx2i * (s[LOCIDX(0, j)] - s[LOCIDX(1, j)]);
250         }
251     }
252     if (wallRight)

```

```

253 {
254     #pragma omp parallel for schedule(static)
255     for (int j = startY; j < stopY; ++j)
256     {
257         v[LOCIDX(Chunkx - 1, j)] = 2.0 * dx2i * (s[LOCIDX(Chunkx - 1, j)] - s[LOCIDX(Chunkx - 2, j)
258             ]);
259     }
260 }
261 // Compute interior vorticity - Obtained using equations 10 on "brief.pdf"
262 #pragma omp parallel for schedule(static) private(tmp1ROW,tmp2ROW)
263 for (int j = startY; j < stopY; ++j)
264 {
265     /**
266     * #pragma omp parallel for schedule(static) collapse(2)
267     * for (int j = startY; j < stopY; ++j)
268     * {
269     * for (int i = startX; i < stopX; ++i)
270     * {
271     * v[LOCIDX(i, j)] = dx2i * (2.0 * s[LOCIDX(i, j)] - s[LOCIDX(i + 1, j)] - s[LOCIDX(i - 1, j)])
272     * //< term 1
273     * + 1.0 / dy / dy * (2.0 * s[LOCIDX(i, j)] - s[LOCIDX(i, j - 1)] - s[LOCIDX(i, j + 1)]); //<
274     * term 2
275     * }
276     * }
277     */
278 tmp1ROW = new double[N]();
279 tmp2ROW = new double[N]();
280 cblas_dscal(N, 0.0, &v[LOCIDX(startX+0,j+0)], 1);
281 cblas_daxpy(N,-1.0,&s[LOCIDX(startX-1,j+0)],1,tmp1ROW,1);
282 cblas_daxpy(N,+2.0,&s[LOCIDX(startX+0,j+0)],1,tmp1ROW,1);
283 cblas_daxpy(N,-1.0,&s[LOCIDX(startX+1,j+0)],1,tmp1ROW,1);
284 cblas_dscal(N,dx2i,tmp1ROW,1);
285
286 cblas_daxpy(N,-1.0,&s[LOCIDX(startX+0,j-1)],1,tmp2ROW,1);
287 cblas_daxpy(N,+2.0,&s[LOCIDX(startX+0,j+0)],1,tmp2ROW,1);
288 cblas_daxpy(N,-1.0,&s[LOCIDX(startX+0,j+1)],1,tmp2ROW,1);
289 cblas_dscal(N,dy2i,tmp2ROW,1);
290 cblas_daxpy(N,1.0,tmp1ROW,1,tmp2ROW,1);
291 cblas_dcopy(N,tmp2ROW,1,&v[LOCIDX(startX+0,j+0)],1);
292 delete[] tmp1ROW;
293 delete[] tmp2ROW;
294 }
295
296 GRID->exchangeGhost(v);
297 // Time advance vorticity - Obtained using equations 11 on "brief.pdf"
298 #pragma omp parallel for schedule(static) private(tmp1ROW,tmp2ROW,tmp3ROW,tmp4ROW,tmp5ROW,tmp6ROW,
299     tmp7ROW)
300 for (int j = startY; j < stopY; ++j)
301 {
302     /**
303     * for (int j = startY; j < stopY; ++j){
304     *
305     * for (int i = startX; i < stopX; ++i)
306     * {
307     * vnew[LOCIDX(i, j)] = v[LOCIDX(i, j)] + dt * (
308     * (
309     * (s[LOCIDX(i + 1, j)] - s[LOCIDX(i - 1, j)]) * 0.5 * dxi * //< Term 1
310     * (v[LOCIDX(i, j - 1)] - v[LOCIDX(i, j + 1)]) * 0.5 * dyi //< Term 2
311     * )
312     * - (
313     * (s[LOCIDX(i, j - 1)] - s[LOCIDX(i, j + 1)]) * 0.5 * dyi * //< Term 3
314     * (v[LOCIDX(i + 1, j)] - v[LOCIDX(i - 1, j)]) * 0.5 * dxi //< Term 4
315     * )

```

```

315     * + nu * (v[LOCIDX(i + 1, j)] - 2.0 * v[LOCIDX(i, j)] + v[LOCIDX(i - 1, j)]) * dx2i ///< Term
316     5
317     * + nu * (v[LOCIDX(i, j - 1)] - 2.0 * v[LOCIDX(i, j)] + v[LOCIDX(i, j + 1)]) * dy2i ///< Term
318     6
319     * );
320     * }
321     * }
322     * */
323 tmp1ROW = new double[N]();
324 tmp2ROW = new double[N]();
325 tmp3ROW = new double[N]();
326 tmp4ROW = new double[N]();
327 tmp5ROW = new double[N]();
328 tmp6ROW = new double[N]();
329 tmp7ROW = new double[N]();
330
331 cblas_dscal(N, 0.0, &vnew[LOCIDX(startX+0,j+0)], 1);
332
333 cblas_daxpy(N, 1.0, &v[LOCIDX(startX+1,j+0)], 1, tmp5ROW, 1);
334 cblas_daxpy(N, 1.0, &v[LOCIDX(startX-1,j+0)], 1, tmp5ROW, 1);
335 cblas_daxpy(N, -2.0, &v[LOCIDX(startX+0,j+0)], 1, tmp5ROW, 1);
336 cblas_dscal(N, nu*dx2i, tmp5ROW, 1);
337
338 cblas_daxpy(N, 1.0, &v[LOCIDX(startX+0,j+1)], 1, tmp6ROW, 1);
339 cblas_daxpy(N, 1.0, &v[LOCIDX(startX+0,j-1)], 1, tmp6ROW, 1);
340 cblas_daxpy(N, -2.0, &v[LOCIDX(startX+0,j+0)], 1, tmp6ROW, 1);
341 cblas_dscal(N, nu*dy2i, tmp6ROW, 1);
342
343 cblas_daxpy(N, 1.0, tmp5ROW, 1, tmp6ROW, 1);
344
345 cblas_daxpy(N, 1.0, &s[LOCIDX(startX+1,j+0)], 1, tmp1ROW, 1);
346 cblas_daxpy(N, -1.0, &s[LOCIDX(startX-1,j+0)], 1, tmp1ROW, 1);
347 cblas_dscal(N, 0.5*dxi, tmp1ROW, 1);
348
349 cblas_daxpy(N, 1.0, &v[LOCIDX(startX+1,j+0)], 1, tmp4ROW, 1);
350 cblas_daxpy(N, -1.0, &v[LOCIDX(startX-1,j+0)], 1, tmp4ROW, 1);
351 cblas_dscal(N, 0.5*dxi, tmp4ROW, 1);
352
353 cblas_daxpy(N, 1.0, &s[LOCIDX(startX+0,j-1)], 1, tmp3ROW, 1);
354 cblas_daxpy(N, -1.0, &s[LOCIDX(startX+0,j+1)], 1, tmp3ROW, 1);
355 cblas_dscal(N, 0.5*dyi, tmp3ROW, 1);
356
357 cblas_daxpy(N, 1.0, &v[LOCIDX(startX+0,j-1)], 1, tmp2ROW, 1);
358 cblas_daxpy(N, -1.0, &v[LOCIDX(startX+0,j+1)], 1, tmp2ROW, 1);
359 cblas_dscal(N, 0.5*dyi, tmp2ROW, 1);
360
361 cblas_dsbsmv(CblasColMajor, CblasLower, N, 0, 1.0, tmp3ROW, 1, tmp4ROW, 1, 0.0, tmp5ROW, 1);
362 cblas_dsbsmv(CblasColMajor, CblasLower, N, 0, 1.0, tmp1ROW, 1, tmp2ROW, 1, 0.0, tmp7ROW, 1);
363 cblas_daxpy(N, -1.0, tmp5ROW, 1, tmp7ROW, 1);
364
365 cblas_daxpy(N, 1.0, tmp6ROW, 1, tmp7ROW, 1);
366
367 cblas_daxpy(N, dt, tmp7ROW, 1, &vnew[LOCIDX(startX+0,j+0)], 1);
368
369 cblas_daxpy(N, 1.0, &v[LOCIDX(startX+0,j+0)], 1, &vnew[LOCIDX(startX+0,j+0)], 1);
370
371 delete[] tmp1ROW;
372 delete[] tmp2ROW;
373 delete[] tmp3ROW;
374 delete[] tmp4ROW;
375 delete[] tmp5ROW;
376 delete[] tmp6ROW;
377 delete[] tmp7ROW;
378 }

```

```

379     cg->Solve(vnew, s, GRID);
380 }
381
382 void LidDrivenCavity::CartInit(int p, int rank, MPI_Comm &comm)
383 {
384     GRID = new prl::gridData(Nx, Ny, p, rank, comm);
385     int size = (GRID->getChunky() + 2) * (GRID->getChunkx() + 2);
386     v = new double[size]();
387     vnew = new double[size]();
388     s = new double[size]();
389 }
390
391 void LidDrivenCavity::CleanUp()
392 {
393     /// if v has been initialised e.g. no longer nullptr free up all the memory as solver has been
394     /// called
395     if (GRID)
396     {
397         delete GRID;
398         /// Stop double deletion
399         GRID = nullptr;
400         delete[] v;
401         delete[] vnew;
402         delete[] s;
403     }
404 }

```

Code Snippet D.1: "LidDrivenCavity.cpp" that implements the domain and parameters of problem

Appendix E

Solver CG

```
1  /**
2   * @file SolverCG.cpp
3   * @author Paul Kallarackel
4   * @brief This file details the methods used to solve the system of equations in space for the lid-
5   *        drigven cavity att a given timestep
6   * @version 0.1
7   * @date 2024-02-28
8   *
9   * @copyright Copyright (c) 2024
10  */
11 #include <iostream>
12 #include <algorithm>
13 #include <cstring>
14 #include <cmath>
15 #include <unistd.h>
16 #include <omp.h>
17 #include <cblas.h>
18 #include "../include/SolverCG.h"
19 #include "../include/ParallelFunc.h"
20 using namespace std;
21
22 SolverCG::SolverCG(int pNx, int pNy, double pdx, double pdy, prl::gridData* GRID)
23 {
24     /// Initialise domain discretisation and state vectors
25     dx = pdx;
26     dy = pdy;
27     Nx = pNx;
28     Ny = pNy;
29     int Chunkx = GRID->getChunkx();
30     int Chunky = GRID->getChunky();
31     int size = (Chunky + 2) * (Chunkx + 2);
32     r = new double[size];
33     p = new double[size];
34     z = new double[size];
35     t = new double[size]; //temp
36 }
37
38
39 SolverCG::~SolverCG()
40 {
41     /// free up memory
42     if (r)
43     {
44         delete[] r;
45         delete[] p;
46         delete[] z;
47         delete[] t;
48     }
49 }
50
51 void SolverCG::Solve(double* b, double* x, prl::gridData* GRID) {
52     int Chunkx = GRID->getChunkx();
53     int Chunky = GRID->getChunky();
54     unsigned int n =(Chunkx+2)*(Chunky+2);
55     int k;
```



```

56 double alpha=0.0;
57 double alphanum=0.0;
58 double alphaden=0.0;
59
60 double localphanum=0.0;
61 double localphaden=0.0;
62 double beta=0.0;
63 double betanum=0.0;
64 double betaden=0.0;
65
66 double locbetanum=0.0;
67 double locbetaden=0.0;
68 double eps; /// current error
69 double loceps = 0.0;; /// current error
70 double tol = 0.001; /// tolerance of solver
71
72 /// Print out current error, calculated using norm-2
73 loceps=0.0;
74 GRID->edgeZero(b);
75
76 loceps = cblas_ddot(n, b, 1,b,1);
77 MPI_Allreduce(&loceps,&eps,1,MPI_DOUBLE,MPI_SUM,MPI_COMM_WORLD);
78 eps=sqrt(eps);
79 if (eps < tol*tol) {
80     std::fill(x, x+n, 0.0);
81     if(GRID->getCenter()==0)cout << "Norm is " << eps << endl;
82     return;
83 }
84
85 /// Initialise conjugate gradient algorithm with correct states and boundary conditions.
86 ApplyOperator(x, t,GRID);
87
88 cblas_dcopy(n, b, 1, r, 1); // r_0 = b (i.e. b)
89
90 ImposeBC(r,GRID);
91
92 cblas_daxpy(n, -1.0, t, 1, r, 1);
93
94
95 /// solve for outer walls
96 Precondition(r, z,GRID);
97
98 cblas_dcopy(n, z, 1, p, 1); // p_0 = r_0
99
100
101 k = 0;
102 /// Using conjugate gradient method to solve for new vorticity
103 do {
104     k++;
105     /// Perform action of  $\nabla^2 * p$ 
106
107     ApplyOperator(p, t,GRID);
108
109     localphaden = 0.0;
110     localphaden = cblas_ddot(n, t, 1, p, 1); //  $\alpha = p_k^T A p_k$ 
111     MPI_Allreduce(&localphaden,&alphaden,1,MPI_DOUBLE,MPI_SUM,MPI_COMM_WORLD);
112
113     localphanum = 0.0;
114     localphanum = cblas_ddot(n, r, 1, z, 1); // compute  $\alpha_k$ 
115     MPI_Allreduce(&localphanum,&alphanum,1,MPI_DOUBLE,MPI_SUM,MPI_COMM_WORLD);
116
117     alpha = alphanum/alphaden;
118     locbetaden = 0.0;
119     locbetaden = cblas_ddot(n, r, 1, z, 1); //  $z_k^T r_k$ 
120     MPI_Allreduce(&locbetaden,&betaden,1,MPI_DOUBLE,MPI_SUM,MPI_COMM_WORLD);
121

```

```

122     cblas_daxpy(n, alpha, p, 1, x, 1); //  $x_{k+1} = x_k + \alpha_k p_k$ 
123
124     cblas_daxpy(n, -alpha, t, 1, r, 1); //  $r_{k+1} = r_k - \alpha_k A p_k$ 
125
126     loceps=0.0;
127     GRID->edgeZero(r);
128     loceps = cblas_ddot(n, r, 1, r, 1);
129     MPI_Allreduce(&loceps, &eps, 1, MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD);
130     eps=sqrt(eps);
131
132     if (eps < tol*tol) {
133         break;
134     }
135     Precondition(r, z, GRID);
136
137     locbetanum = 0.0;
138     locbetanum = cblas_ddot(n, r, 1, z, 1);
139     MPI_Allreduce(&locbetanum, &betanum, 1, MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD);
140     beta = betanum/betaden;
141
142     cblas_dcopy(n, z, 1, t, 1);
143
144     cblas_daxpy(n, beta, p, 1, t, 1);
145
146     cblas_dcopy(n, t, 1, p, 1);
147
148 } while (k < 5000); // Set a maximum number of iterations
149
150 if (k == 5000) {
151     if(GRID->getCenter()==0)cout << "FAILED TO CONVERGE" << endl;
152     exit(-1);
153 }
154
155 if(GRID->getCenter()==0)cout << "Converged in " << k << " iterations. eps = " << eps << endl;
156 }
157
158 void SolverCG::ApplyOperator(double* in, double* out, prl::gridData* GRID) {
159     double dx2i = 1.0/dx/dx;
160     double dy2i = 1.0/dy/dy;
161     int Chunkx = GRID->getChunkx();
162     int Chunky = GRID->getChunky();
163     GRID->exchangeGhost(in);
164     bool wallTop = GRID->getUp()<0;
165     bool wallBottom = GRID->getDown()<0;
166     bool wallLeft = GRID->getLeft()<0;
167     bool wallRight = GRID->getRight()<0;
168     int startY = (!wallTop ? 0 : 1);
169     int startX = (!wallLeft ? 0 : 1);
170     int stopY = (!wallBottom ? Chunky : Chunky - 1);
171     int stopX = (!wallRight ? Chunkx : Chunkx - 1);
172     int N = stopX-startX;
173     double *tmpROW = nullptr;
174
175     #pragma omp parallel for schedule(static) private(tmpROW)
176     for (int j = startY; j < stopY; ++j)
177     {
178         tmpROW = new double[N]();
179         /**
180          * (- in[LOCIDX(i - 1, j)]
181          * + 2.0 * in[LOCIDX(i, j)]
182          * - in[LOCIDX(i + 1, j)]) * dx2i
183          *
184          */
185         cblas_dscal(N, 0.0, &out[LOCIDX(startX+0,j+0)], 1);
186         cblas_daxpy(N, -1.0, &in[LOCIDX(startX-1,j+0)], 1, tmpROW, 1);
187         cblas_daxpy(N, +2.0, &in[LOCIDX(startX+0,j+0)], 1, tmpROW, 1);

```

```

188     cblas_daxpy(N,-1.0,&in[LOCIDX(startX+1,j+0)],1,tmpROW,1);
189     cblas_dscal(N,dx2i,tmpROW,1);
190
191     /**
192     * ( - in[LOCIDX(i, j-1)]
193     * + 2.0 * in[LOCIDX(i, j)]
194     * - in[LOCIDX(i, j+1)]) * dy2i;
195     *
196     */
197     cblas_daxpy(N,-1.0,&in[LOCIDX(startX+0,j-1)],1,&out[LOCIDX(startX+0,j+0)],1);
198     cblas_daxpy(N,+2.0,&in[LOCIDX(startX+0,j+0)],1,&out[LOCIDX(startX+0,j+0)],1);
199     cblas_daxpy(N,-1.0,&in[LOCIDX(startX+0,j+1)],1,&out[LOCIDX(startX+0,j+0)],1);
200     cblas_dscal(N,dy2i,&out[LOCIDX(startX+0,j+0)],1);
201
202     ///out[LOCIDX(i, j)] = (- in[LOCIDX(i - 1, j)]+ 2.0 * in[LOCIDX(i, j)]- in[LOCIDX(i + 1, j)]) *
203     dx2i + ( - in[LOCIDX(i, j-1)]+ 2.0 * in[LOCIDX(i, j)]- in[LOCIDX(i, j+1)]) * dy2i;
204     cblas_daxpy(N,1.0,tmpROW,1,&out[LOCIDX(startX+0,j+0)],1);
205     delete[] tmpROW;
206 }
207
208 void SolverCG::Precondition(double* in, double* out,prl::gridData* GRID) {
209     int Chunkx = GRID->getChunkx();
210     int Chunky = GRID->getChunky();
211     bool wallTop = GRID->getUp()<0;
212     bool wallBottom = GRID->getDown()<0;
213     bool wallLeft = GRID->getLeft()<0;
214     bool wallRight =GRID->getRight()<0;
215     double dx2i = 1.0/dx/dx;
216     double dy2i = 1.0/dy/dy;
217     double factor = 2.0*(dx2i + dy2i);
218     int startY = (!wallTop ? 0 : 1);
219     int startX = (!wallLeft ? 0 : 1);
220     int stopY = (!wallBottom ? Chunky : Chunky - 1);
221     int stopX = (!wallRight ? Chunkx : Chunkx - 1);
222
223     #pragma omp parallel for schedule(static)
224     for (int j = startY; j < stopY; ++j)
225     {
226         /**
227         * #pragma omp parallel for schedule(static)
228         * for (int j = (!wallTop ? 0 : 1); j < (!wallBottom ? Chunky : Chunky - 1); ++j)
229         * {
230         * for (int i = (!wallLeft ? 0 : 1); i < (!wallRight ? Chunkx : Chunkx - 1); ++i)
231         * {
232         *
233         * out[LOCIDX(i, j)] = in[LOCIDX(i, j)] /factor;
234         * }
235         * }
236         */
237         cblas_dcopy(stopX-startX,&in[LOCIDX(startX, j)],1,&out[LOCIDX(startX, j)],1);
238         cblas_dscal(stopX-startX,1.0/factor,&out[LOCIDX(startX, j)],1);
239     }
240     if (wallTop)
241     {
242         #pragma omp parallel for schedule(static)
243         for (int i = 0; i < Chunkx; ++i)
244         {
245             out[LOCIDX(i, 0)] = in[LOCIDX(i, 0)];
246         }
247     }
248     if (wallBottom)
249     {
250         #pragma omp parallel for schedule(static)
251         for (int i = 0; i < Chunkx; ++i)
252         {

```

```

253     out[LOCIDX(i, Chunky - 1)] = in[LOCIDX(i, Chunky - 1)];
254 }
255 }
256 if (wallLeft)
257 {
258     #pragma omp parallel for schedule(static)
259     for (int j = 0; j < Chunky; ++j)
260     {
261         out[LOCIDX(0, j)] = in[LOCIDX(0, j)];
262     }
263 }
264 if (wallRight)
265 {
266     #pragma omp parallel for schedule(static)
267     for (int j = 0; j < Chunky; ++j)
268     {
269         out[LOCIDX(Chunkx - 1, j)] = in[LOCIDX(Chunkx - 1, j)];
270     }
271 }
272 }
273 void SolverCG::ImposeBC(double* inout, prl::gridData* GRID) {
274     int Chunkx = GRID->getChunkx();
275     int Chunky = GRID->getChunky();
276     bool wallTop = GRID->getUp() < 0;
277     bool wallBottom = GRID->getDown() < 0;
278     bool wallLeft = GRID->getLeft() < 0;
279     bool wallRight = GRID->getRight() < 0;
280     // Boundaries
281     if (wallTop)
282     {
283
284         #pragma omp parallel for schedule(static)
285         for (int i = 0; i < Chunkx; ++i)
286         {
287             inout[LOCIDX(i, 0)] = 0.0;
288         }
289     }
290     if (wallBottom)
291     {
292         #pragma omp parallel for schedule(static)
293         for (int i = 0; i < Chunkx; ++i)
294         {
295             inout[LOCIDX(i, Chunky - 1)] = 0.0;
296         }
297     }
298     if (wallLeft)
299     {
300         #pragma omp parallel for schedule(static)
301         for (int j = 0; j < Chunky; ++j)
302         {
303             inout[LOCIDX(0, j)] = 0.0;
304         }
305     }
306     if (wallRight)
307     {
308         #pragma omp parallel for schedule(static)
309         for (int j = 0; j < Chunky; ++j)
310         {
311             inout[LOCIDX(Chunkx - 1, j)] = 0.0;
312         }
313     }
314 }

```

Code Snippet E.1: “SolverCG.cpp” that implements the solve of $-\nabla^2 \psi \hat{\mathbf{k}} = \omega$

MATLAB Data Analysis

```
1 clear; clc; close all;
2 screen_ratio=disp_transform(10,10)./10;
3 xratio=screen_ratio(1);
4 yratio=screen_ratio(2);
5 % Read the data from the text file
6 files = [...
7     "timingsLx1Ly1Nx101Ny101Re500dt0_005T5",...
8     "timingsLx1Ly1Nx201Ny201Re1000dt0_005T5"
9 ];
10
11
12 for i = 1:length(files)
13     file = files(i);
14
15     % file = "timingsLx1Ly1Nx201Ny201Re10dt0_00005T0_005";
16     data = readmatrix(strcat(file, '.txt'));
17
18
19     % Extract relevant data
20     MPI_ranks = data(:,1);
21     OMP_threads = data(:,2);
22     INIT_time = data(:,3);
23     IC_time = data(:,4);
24     SOLVE_time = data(:,5);
25     FINAL_time = data(:,6);
26     Total_time = data(:,7);
27
28     IDX = 5;
29     n=2;
30     FIGSIZE=[1000,585,757.6666666666665,652.6666666666665];
31     FIGSIZE(1) = FIGSIZE(1) *xratio;
32     FIGSIZE(2) = FIGSIZE(2) *yratio;
33     FIGSIZE(3) = FIGSIZE(3) *xratio;
34     FIGSIZE(4) = FIGSIZE(4) *yratio;
35
36     % MPI V Total
37     mpitotal = figure('Position',FIGSIZE);
38     p1=plot(MPI_ranks(1:IDX-1), Total_time(1:IDX-1), 'r-x', 'MarkerSize', 10); % Red cross
39     markers with increased size
40     xlabel('MPI Ranks');
41     ylabel('Total Time [$s$]');
42     hold on;
43     % Add curved line of best fit (second-order polynomial)
44     % %p=polyfit(MPI_ranks(1:IDX-1), Total_time(1:IDX-1), n); % Second-order polynomial
45     % x_fit = min(MPI_ranks(1:IDX-1)):0.1:max(MPI_ranks(1:IDX-1));
46     % y_fit = spline(p, x_fit);
47     % %plot(x_fit, y_fit, 'r--');
48
```

```

49
50 % OMP V Total
51 omptotal = figure('Position',FIGSIZE);
52 p2=plot(OMP_threads(IDX:end), Total_time(IDX:end), 'r-x', 'MarkerSize', 10); % Red cross
    markers with increased size
53 xlabel('Open MP Threads');
54 ylabel('Total Time [$$]');
55 hold on;
56 % Add curved line of best fit (second-order polynomial)
57 % p=polyfit(OMP_threads(IDX:end), Total_time(IDX:end), n); % Second-order polynomial
58 % x_fit = min(OMP_threads(IDX:end)):0.1:max(OMP_threads(IDX:end));
59 % y_fit = spline(p, x_fit);
60 % %plot(x_fit, y_fit, 'r--');
61
62
63 % MPI V INIT
64 mpiinit = figure('Position',FIGSIZE);
65 p1=plot(MPI_ranks(1:IDX-1), INIT_time(1:IDX-1), 'b-x', 'MarkerSize', 10); % Red cross
    markers with increased size
66 xlabel('MPI Ranks');
67 ylabel('Initialising Time [$$]');
68 hold on;
69 % Add curved line of best fit (second-order polynomial)
70 % p=polyfit(MPI_ranks(1:IDX-1), INIT_time(1:IDX-1), n); % Second-order polynomial
71 % x_fit = min(MPI_ranks(1:IDX-1)):0.1:max(MPI_ranks(1:IDX-1));
72 % y_fit = spline(p, x_fit);
73 % %plot(x_fit, y_fit, 'r--');
74
75
76 % OMP V INIT
77 ompinit = figure('Position',FIGSIZE);
78 p2=plot(OMP_threads(IDX:end), INIT_time(IDX:end), 'b-x', 'MarkerSize', 10); % Red cross
    markers with increased size
79 xlabel('Open MP Threads');
80 ylabel('Initialising Time [$$]');
81 hold on;
82 % Add curved line of best fit (second-order polynomial)
83 % p=polyfit(OMP_threads(IDX:end), INIT_time(IDX:end), n); % Second-order polynomial
84 % x_fit = min(OMP_threads(IDX:end)):0.1:max(OMP_threads(IDX:end));
85 % y_fit = spline(p, x_fit);
86 % %plot(x_fit, y_fit, 'r--');
87
88
89 % MPI V IC
90 mpiic = figure('Position',FIGSIZE);
91 p1=plot(MPI_ranks(1:IDX-1), IC_time(1:IDX-1), 'g-x', 'MarkerSize', 10); % Red cross
    markers with increased size
92 xlabel('MPI Ranks');
93 ylabel('Writing Initial Conditions Time [$$]');
94 hold on;
95 % Add curved line of best fit (second-order polynomial)
96 % p=polyfit(MPI_ranks(1:IDX-1), IC_time(1:IDX-1), n); % Second-order polynomial
97 % x_fit = min(MPI_ranks(1:IDX-1)):0.1:max(MPI_ranks(1:IDX-1));
98 % y_fit = spline(p, x_fit);
99 % %plot(x_fit, y_fit, 'r--');
100
101
102 % OMP V IC

```

```

103 ompic = figure('Position',FIGSIZE);
104 p2=plot(OMP_threads(IDX:end), IC_time(IDX:end), 'g-x', 'MarkerSize', 10); % Red cross
    markers with increased size
105 xlabel('Open MP Threads');
106 ylabel('Writing Initial Conditions Time [$s$]');
107 hold on;
108 % Add curved line of best fit (second-order polynomial)
109 % %p=polyfit(OMP_threads(IDX:end), IC_time(IDX:end), n); % Second-order polynomial
110 % x_fit = min(OMP_threads(IDX:end)):0.1:max(OMP_threads(IDX:end));
111 % y_fit = spline(p, x_fit);
112 % %plot(x_fit, y_fit, 'r--');
113
114
115 % MPI V SOLVE
116 mpisolve = figure('Position',FIGSIZE);
117 p1=plot(MPI_ranks(1:IDX-1), SOLVE_time(1:IDX-1), 'k-x', 'MarkerSize', 10); % Red cross
    markers with increased size
118 xlabel('MPI Ranks');
119 ylabel('Solve Time [$s$]');
120 hold on;
121 % Add curved line of best fit (second-order polynomial)
122 % %p=polyfit(MPI_ranks(1:IDX-1), SOLVE_time(1:IDX-1), n); % Second-order polynomial
123 % x_fit = min(MPI_ranks(1:IDX-1)):0.1:max(MPI_ranks(1:IDX-1));
124 % y_fit = spline(p, x_fit);
125 % %plot(x_fit, y_fit, 'r--');
126
127
128 % OMP V SOLVE
129 ompsolve =figure('Position',FIGSIZE);
130 p2=plot(OMP_threads(IDX:end), SOLVE_time(IDX:end), 'k-x', 'MarkerSize', 10); % Red cross
    markers with increased size
131 xlabel('Open MP Threads');
132 ylabel('Solve Time [$s$]');
133 hold on;
134 % Add curved line of best fit (second-order polynomial)
135 % %p=polyfit(OMP_threads(IDX:end), SOLVE_time(IDX:end), n); % Second-order polynomial
136 % x_fit = min(OMP_threads(IDX:end)):0.1:max(OMP_threads(IDX:end));
137 % y_fit = spline(p, x_fit);
138 % %plot(x_fit, y_fit, 'r--');
139
140
141 % MPI V FINAL
142 mpifinal =figure('Position',FIGSIZE);
143 p1=plot(MPI_ranks(1:IDX-1), FINAL_time(1:IDX-1), 'm-x', 'MarkerSize', 10); % Red cross
    markers with increased size
144 xlabel('MPI Ranks');
145 ylabel('Write Final Time [$s$]');
146 hold on;
147 % Add curved line of best fit (second-order polynomial)
148 % %p=polyfit(MPI_ranks(1:IDX-1), FINAL_time(1:IDX-1), n); % Second-order polynomial
149 % x_fit = min(MPI_ranks(1:IDX-1)):0.1:max(MPI_ranks(1:IDX-1));
150 % y_fit = spline(p, x_fit);
151 % %plot(x_fit, y_fit, 'r--');
152
153
154 % OMP V FINAL
155 ompfinal =figure('Position',FIGSIZE);

```

```

156 p2=plot(OMP_threads(IDX:end), FINAL_time(IDX:end), 'm-x', 'MarkerSize', 10); % Red cross
    markers with increased size
157 xlabel('Open MP Threads');
158 ylabel('Write Final Time [$s$]');
159 hold on;
160 % Add curved line of best fit (second-order polynomial)
161 % %p=polyfit(OMP_threads(IDX:end), FINAL_time(IDX:end), n); % Second-order polynomial
162 % x_fit = min(OMP_threads(IDX:end)):0.1:max(OMP_threads(IDX:end));
163 % y_fit = spline(p, x_fit);
164 % %plot(x_fit, y_fit, 'r--');
165
166 % MPI V normalise time
167 mpinormal =figure('Position',FIGSIZE);
168 % normal = Total_time(1:IDX-1)./max(Total_time(1:IDX-1));
169 % p3=plot(MPI_ranks(1:IDX-1), normal, 'rx', 'MarkerSize', 10); % Red cross markers with
    increased size
170 xlabel('MPI Ranks');
171 ylabel('Normalised Time');
172
173 hold on;
174 % Add curved line of best fit (second-order polynomial)
175 % %p=polyfit(MPI_ranks(1:IDX-1), normal, n); % Second-order polynomial
176 % x_fit = min(MPI_ranks(1:IDX-1)):0.1:max(MPI_ranks(1:IDX-1));
177 % y_fit = spline(p, x_fit);
178 % %plot(x_fit, y_fit, 'r--');
179
180 normal = INIT_time(1:IDX-1)./max(INIT_time(1:IDX-1));
181 p4=plot(MPI_ranks(1:IDX-1), normal, 'b-x', 'MarkerSize', 10); % Red cross markers with
    increased size
182 hold on;
183 % Add curved line of best fit (second-order polynomial)
184 %p=polyfit(MPI_ranks(1:IDX-1), normal, n); % Second-order polynomial
185 x_fit = min(MPI_ranks(1:IDX-1)):0.1:max(MPI_ranks(1:IDX-1));
186 y_fit = spline(MPI_ranks(1:IDX-1),normal,x_fit);
187 %plot(x_fit, y_fit, 'b--');
188
189 normal = IC_time(1:IDX-1)./max(IC_time(1:IDX-1));
190 p5=plot(MPI_ranks(1:IDX-1), normal, 'g-x', 'MarkerSize', 10); % Red cross markers with
    increased size
191 hold on;
192 % Add curved line of best fit (second-order polynomial)
193 %p=polyfit(MPI_ranks(1:IDX-1), normal, n); % Second-order polynomial
194 x_fit = min(MPI_ranks(1:IDX-1)):0.1:max(MPI_ranks(1:IDX-1));
195 y_fit = spline(MPI_ranks(1:IDX-1),normal,x_fit);
196 %plot(x_fit, y_fit, 'g--');
197
198 normal = SOLVE_time(1:IDX-1)./max(SOLVE_time(1:IDX-1));
199 p6=plot(MPI_ranks(1:IDX-1), normal, 'k-x', 'MarkerSize', 10); % Red cross markers with
    increased size
200 hold on;
201 % Add curved line of best fit (second-order polynomial)
202 %p=polyfit(MPI_ranks(1:IDX-1), normal, n); % Second-order polynomial
203 x_fit = min(MPI_ranks(1:IDX-1)):0.1:max(MPI_ranks(1:IDX-1));
204 y_fit = spline(MPI_ranks(1:IDX-1),normal,x_fit);
205 %plot(x_fit, y_fit, 'k--');
206
207 normal = FINAL_time(1:IDX-1)./max(FINAL_time(1:IDX-1));

```



```

208 p7=plot(MPI_ranks(1:IDX-1), normal, 'm-x', 'MarkerSize', 10); % Red cross markers with
    increased size
209 hold on;
210 % Add curved line of best fit (second-order polynomial)
211 %p=polyfit(MPI_ranks(1:IDX-1), normal, n); % Second-order polynomial
212 x_fit = min(MPI_ranks(1:IDX-1)):0.1:max(MPI_ranks(1:IDX-1));
213 y_fit = spline(MPI_ranks(1:IDX-1),normal,x_fit);
214 %plot(x_fit, y_fit, 'm--');
215
216 % Get the current y-axis limits
217 current_ylim = ylim;
218
219 % Set the lower limit to 0 and keep the upper limit unchanged
220 ylim([0, current_ylim(2)]);
221 legend([p4,p5,p6,p7],["Initialise","Write Initial","Solve","Write Final"])
222
223
224 % OMP V normalise time
225 ompnormal =figure('Position',FIGSIZE);
226 % normal = Total_time(IDX:end)./max(Total_time(IDX:end));
227 % p3=plot(OMP_threads(IDX:end), normal, 'r-x', 'MarkerSize', 10); % Red cross markers
    with increased size
228 xlabel('OMP threads');
229 ylabel('Normalised Time');
230
231 hold on;
232 % Add curved line of best fit (second-order polynomial)
233 % %p=polyfit(OMP_threads(IDX:end), normal, n); % Second-order polynomial
234 % x_fit = min(OMP_threads(IDX:end)):0.1:max(OMP_threads(IDX:end));
235 % y_fit = spline(OMP_threads(IDX:end),normal,x_fit);
236 % %plot(x_fit, y_fit, 'r--');
237
238 normal = INIT_time(IDX:end)./max(INIT_time(IDX:end));
239 p4=plot(OMP_threads(IDX:end), normal, 'b-x', 'MarkerSize', 10); % Red cross markers with
    increased size
240 hold on;
241 % Add curved line of best fit (second-order polynomial)
242 %p=polyfit(OMP_threads(IDX:end), normal, n); % Second-order polynomial
243 x_fit = min(OMP_threads(IDX:end)):0.1:max(OMP_threads(IDX:end));
244 y_fit = spline(OMP_threads(IDX:end),normal,x_fit);
245 %plot(x_fit, y_fit, 'b--');
246
247 normal = IC_time(IDX:end)./max(IC_time(IDX:end));
248 p5=plot(OMP_threads(IDX:end), normal, 'g-x', 'MarkerSize', 10); % Red cross markers with
    increased size
249 hold on;
250 % Add curved line of best fit (second-order polynomial)
251 %p=polyfit(OMP_threads(IDX:end), normal, n); % Second-order polynomial
252 x_fit = min(OMP_threads(IDX:end)):0.1:max(OMP_threads(IDX:end));
253 y_fit = spline(OMP_threads(IDX:end),normal,x_fit);
254 %plot(x_fit, y_fit, 'g--');
255
256 normal = SOLVE_time(IDX:end)./max(SOLVE_time(IDX:end));
257 p6=plot(OMP_threads(IDX:end), normal, 'k-x', 'MarkerSize', 10); % Red cross markers with
    increased size
258 hold on;
259 % Add curved line of best fit (second-order polynomial)
260 %p=polyfit(OMP_threads(IDX:end), normal, n); % Second-order polynomial

```

```

261 x_fit = min(OMP_threads(IDX:end)):0.1:max(OMP_threads(IDX:end));
262 y_fit = spline(OMP_threads(IDX:end),normal,x_fit);
263 %plot(x_fit, y_fit, 'k--');
264
265 normal = FINAL_time(IDX:end)./max(FINAL_time(IDX:end));
266 p7=plot(OMP_threads(IDX:end), normal, 'm-x', 'MarkerSize', 10); % Red cross markers with
    increased size
267 hold on;
268 % Add curved line of best fit (second-order polynomial)
269 %p=polyfit(OMP_threads(IDX:end), normal, n); % Second-order polynomial
270 x_fit = min(OMP_threads(IDX:end)):0.1:max(OMP_threads(IDX:end));
271 y_fit = spline(OMP_threads(IDX:end),normal,x_fit);
272 %plot(x_fit, y_fit, 'm--');
273
274 current_ylim = ylim;
275 ylim([0,current_ylim(2)]);
276 legend([p4,p5,p6,p7],["Initialise","Write Initial","Solve","Write Final"])
277
278
279
280
281 prettier();
282
283
284 figure(mpitotal);
285 currxlim = xlim;
286 xlim([1,currxlim(2)]);
287 saveas(gcf,strcat("../Images/",file,"__MPIVTOTAL"),"png");
288 figure(omptotal);
289 currxlim = xlim;
290 xlim([1,currxlim(2)]);
291 saveas(gcf,strcat("../Images/",file,"__OMPVTOTAL"),"png");
292 figure(mpiinit);
293 currxlim = xlim;
294 xlim([1,currxlim(2)]);
295 saveas(gcf,strcat("../Images/",file,"__MPIVINIT"),"png");
296 figure(ompinit);
297 currxlim = xlim;
298 xlim([1,currxlim(2)]);
299 saveas(gcf,strcat("../Images/",file,"__OMPVINIT"),"png");
300 figure(mpiic);
301 currxlim = xlim;
302 xlim([1,currxlim(2)]);
303 saveas(gcf,strcat("../Images/",file,"__MPIVIC"),"png");
304 figure(ompic);
305 currxlim = xlim;
306 xlim([1,currxlim(2)]);
307 saveas(gcf,strcat("../Images/",file,"__OMPVIC"),"png");
308 figure(mpisolve);
309 currxlim = xlim;
310 xlim([1,currxlim(2)]);
311 saveas(gcf,strcat("../Images/",file,"__MPIVSOLVE"),"png");
312 figure(ompsolve);
313 currxlim = xlim;
314 xlim([1,currxlim(2)]);
315 saveas(gcf,strcat("../Images/",file,"__OMPVSOLVE"),"png");
316 figure(mpifinal);
317 currxlim = xlim;

```

```

318     xlim([1,currxlim(2)]);
319     saveas(gcf,strcat("../Images/",file,"__MPIVFINAL"),"png");
320     figure(ompfinal);
321     currxlim = xlim;
322     xlim([1,currxlim(2)]);
323     saveas(gcf,strcat("../Images/",file,"__OMPVFINAL"),"png");
324     figure(mpinormal);
325     currxlim = xlim;
326     xlim([1,currxlim(2)]);
327     saveas(gcf,strcat("../Images/",file,"__MPIVNORMAL"),"png");
328     figure(ompnormal);
329     currxlim = xlim;
330     xlim([1,currxlim(2)]);
331     saveas(gcf,strcat("../Images/",file,"__OMPVNORMAL"),"png");
332
333     close all;
334 end

```

Code Snippet F.1: “myPlot.m” script to generate plots for report from sample “timings.txt” as seen in Figure A.2

```

1  #!/bin/bash
2  # Define parameters
3  Nt=1
4  export OMP_NUM_THREADS=$Nt
5  Lx=1
6  Ly=1
7  Nx=201
8  Ny=201
9  Re=1000
10 dt=0.001
11 T=5
12 # Write problem settings to a file
13 echo "Problem settings:" > timings.txt
14 echo "Lx=$Lx Ly=$Ly Nx=$Nx Ny=$Ny Re=$Re dt=$dt T=$T" >> timings.txt
15 echo "MPIRANKS [#] | OMP THREADS [#] | Initialising [s] | WriteIC [s] | Integrate [s] | WriteSOL [s] |
   ↳ Total [s]" >> timings.txt
16 echo "-----"
17 # Loop through square numbers from 1 to 4
18 for (( i=1; i<=4; i=i+1 ))
19 do
20     Np=$(( $i*$i ))
21     echo "Running with $Np MPI ranks"
22     echo "Running with $Nt OMP threads"
23     # Run the program
24     mpirun --bind-to none -np $Np ../../bin/solver --Lx $Lx --Ly $Ly --Nx $Nx --Ny $Ny --Re $Re --dt $dt
   ↳ --T $T
25     echo "-----"
26 done
27 # Reset Np
28 Np=1
29 # Loop through OMP threads from 1 to 16
30 Nt=1
31 echo "Running with $Np MPI ranks"
32 echo "Running with $Nt OMP threads"
33 export OMP_NUM_THREADS=$Nt
34 # Run the program
35 mpirun --bind-to none -np $Np ../../solver --Lx $Lx --Ly $Ly --Nx $Nx --Ny $Ny --Re $Re --dt $dt --T $T
36 echo "-----"
37 for (( i=1; i<=8; i=i+1 ))
38 do
39     Nt=$(( $i*2 ))
40     echo "Running with $Np MPI ranks"
41     echo "Running with $Nt OMP threads"
42     export OMP_NUM_THREADS=$Nt
43     # Run the program
44     mpirun --bind-to none -np $Np ../../bin/solver --Lx $Lx --Ly $Ly --Nx $Nx --Ny $Ny --Re $Re --dt $dt --T $T
45     echo "-----"
46 done

```

Code Snippet F.2: “run_solver.sh” script to generate scaling data for sample “timings.txt” as seen in Figure A.2

```

1  function prettier()
2  %%%CHOOSE DESIGN
3  LINEWIDTH = 3;
4  MARKERSIZE = 10;
5  LEGENED_FONTSIZE = 15;
6  AXES_FONTSIZE = 20;
7
8
9
10 all_figures = findobj('Type', 'figure');
11 for i = 1:length(all_figures)
12     % Select the figure by its handle
13     figure(all_figures(i));
14     fig = gcf;
15     graph_pretty(fig,LINWIDTH,MARKERSIZE,LEGENED_FONTSIZE,AXES_FONTSIZE)

```

```

16 end
17 end
18
19
20 function graph_pretty (fig,LINWIDTH,MARKERSIZE,LEGENDED_FONTSIZE,AXES_FONTSIZE)
21 % Get the handle of the current figure
22 % fig = gcf;
23
24 % Get the handles of all the axes in the figure
25 axesHandles = findall(fig, 'type', 'axes');
26
27 % Iterate through each axes
28 for i = 1:numel(axesHandles)
29     % Get the handle of the current axes
30     a = axesHandles(i);
31     a.FontSize = LEGENDED_FONTSIZE;
32     % a.Legend.FontSize = 15;
33     % a.Legend.Location = 'Best';
34
35     a.Title.FontWeight = 'Bold';
36     a.Title.Interpreter = 'latex';
37     a.YLabel.FontSize = AXES_FONTSIZE;
38     a.YLabel.FontWeight = 'Bold';
39     a.YLabel.Interpreter = 'latex';
40     a.XLabel.FontSize = AXES_FONTSIZE;
41     a.XLabel.FontWeight = 'Bold';
42     a.XLabel.Interpreter = 'latex';
43     box on;
44     a.LineWidth = LINewidth/2;
45     % Loop through the children elements and check their names
46     for i = 1:height(a.Children)
47         if strcmp(a.Children(i).Type, 'line')
48             a.Children(i).LineWidth = LINewidth;
49             a.Children(i).MarkerSize = MARKERSIZE;
50         end
51     end
52     a.XMinorGrid = 'on';
53     a.XMinorTick = 'on';
54     a.YMinorGrid = 'on';
55     a.YMinorTick = 'on';
56     a.XTickLabelMode = 'auto';
57
58
59
60 end
61 for i =1:height(fig.Children(1))
62     leg = fig.Children(1);
63     if isprop(leg,"Location")
64         leg.Location = 'best';
65         leg.FontSize = LEGENDED_FONTSIZE;
66         leg.FontWeight = 'Bold';
67         leg.Interpreter = 'latex';
68     end
69
70
71
72 end
73

```

```

74 plot_num = flip(fig.Children(1).Children);
75 plotsavail = [1:1:height(plot_num)];
76
77 % % % % % choose which plots to keep
78 % plotstokeep = [1];
79 %
80 %
81 %
82 % match_plot = ~ismember(plotsavail, plotstokeep);
83 % match_plot = match_plot.*plotsavail;
84 % match_plot = match_plot(match_plot ~= 0);
85 % for i = match_plot
86 % delete(plot_num(i));
87 % end
88 %
89 %
90 % plotstokeep = flip(plotstokeep);
91 % prevy = 0;
92 % for i = plotstokeep
93 % plot_num(i).Units = 'normalized';
94 % plot_num(i).Position = [0 prevy 1 1/length(plotstokeep)];
95 % prevy = prevy + 1/length(plotstokeep);
96 %
97 % end
98
99 end

```

Code Snippet F.2: “prettier.m” script to help generate plots for report